AI Tools and the Performance of Large Models

The large-model landscape, the agent tool stack, and third-party evaluation

AI Foundation | Supplementary Lecture 2 | Non-major students

Learning Objectives

1. Describe the industry structure, company backgrounds and business models of the major LLM vendors; judge their openness and supply risk.
2. Trace the history of LLMs and AI agents; understand which bottleneck each stage solved.
3. Master the classification dimensions (modality, architecture, openness, scale, price tier) and pick a class for a given task.
4. Know the major third-party evaluators (LMArena, Artificial Analysis, HELM, Epoch AI, SEAL, OpenCompass, SuperCLUE, etc.) and how they score.
5. Read leaderboards and price sheets correctly; spot data contamination, methodology gaps and value-for-money knees.
6. Argue one model-selection decision with the "capability–price" and "total cost of ownership" frameworks.

Roadmap: eleven sections, industry to selection

No.	Topic	Question answered
1	Comparison coordinates	Why “strongest” is not “most suitable”?
2	Industry map and companies	Who builds models, how do they earn, open or closed?
3	History of large models	Which paradigms led from Transformer to reasoning models?
4	History of agents and tools	How did “able to talk” become “able to work”?
5	LLM taxonomy	Given a task, which class of model should you pick?
6	Third-party evaluators	Who evaluates, with what mechanism, and what are the latest results?
7	Methodology and pitfalls	How can you read a leaderboard without being misled?
8	Pricing and cost engineering	How does a per-million-token price become a per-task cost?
9	Direction-by-direction review	Where is the sweet spot of each of the thirteen directions?
10	Selection methodology	How do you make an auditable selection decision?
11	Practice, risks, summary	Compare hands-on once, and know the limits of the conclusion.

01

Capability, Cost, Risk: Building the Comparison Frame

Three procurement perspectives

Choosing a model is not choosing "the strongest" — it is choosing "the most suitable" under constraints.

- **Capability**: peak performance, stability, context length, tool use, multimodality.
- **Cost**: unit price, token efficiency, retry rate, and indirect costs of latency.
- **Risk**: hallucination, compliance, data residency, vendor lock-in, service shutdown.

Key Terms

Benchmark
Pareto Frontier
Total Cost of Ownership
Hallucination

Example | The same contract-extraction task can cost over 10x more on a flagship tier than a light tier, while accuracy differs by only a few percentage points (illustrative). How much to spend depends on what one failure costs.

A Model Is Not a Standard Commodity

The same base model can differ by tens of percentage points in measured performance across "shells".

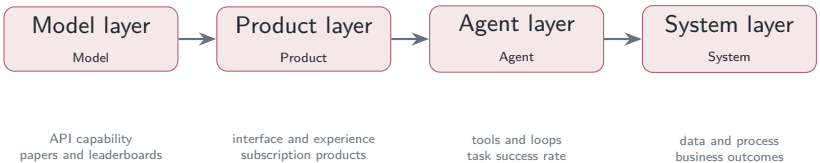
- The base model only supplies generation and judgment ability.
- The harness determines prompt structure, tools, retrieval, retries and validation.
- A leaderboard score therefore cannot be equated with a product's capability.
- Before comparing, declare: are you comparing models, or products and systems?

Key Terms

Base Model
Harness
Agent
Tool Calling

Formula | Task value $V = \Delta A \times N - \Delta C \times N$: a capability gain ΔA must be multiplied by usage frequency N to become value; cost gaps are amplified by frequency the same way. High-frequency tasks are price-sensitive; low-frequency tasks are capability-sensitive.

Four Levels of Comparison: the Lower, the Closer to Real Experience



The same model can yield opposite conclusions at different levels: confirm the level first

Why "No. 1 on the Board" Is Not "the Best Fit"

Leaderboards measure average capability, while your task has specific constraints.

- Leaderboard samples are mostly general knowledge and contest problems, not your task distribution.
- When a ranking gap is inside statistical error, it is not a capability difference.
- Price, latency, context and data compliance can matter more than 1–2 percentage points.
- Correct use: leaderboards shortlist candidates; your own evaluation makes the final call.

Key Terms

Data Contamination
Confidence Interval
Golden Set
Shortlisting

Example | When two models differ by 1 point with overlapping confidence intervals, investing the budget in prompt engineering and retrieval quality usually pays back more than "switching to a stronger model".

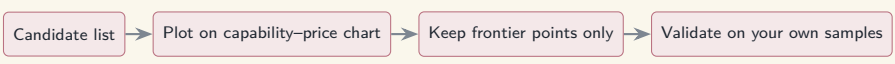
The capability–price Pareto frontier

The real question is not "who is strongest" but "who is strongest at a given budget".

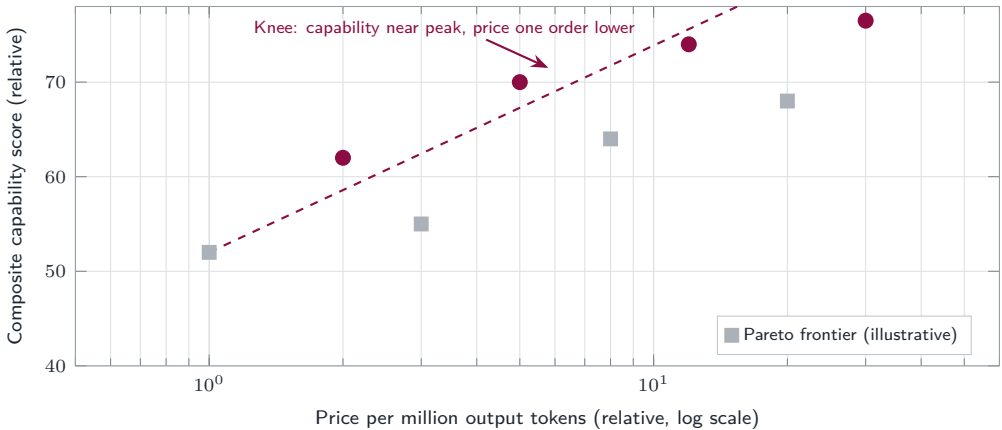
- Plot each model's capability score and per-million-token price on one chart.
- Models on the Pareto frontier cannot improve on one axis without sacrificing the other.
- Models below the frontier usually have no reason to exist — unless latency, compliance or deployment constraints apply.
- Practical selection clusters near the "knee": capability near peak at one order of magnitude lower price.

Key Terms

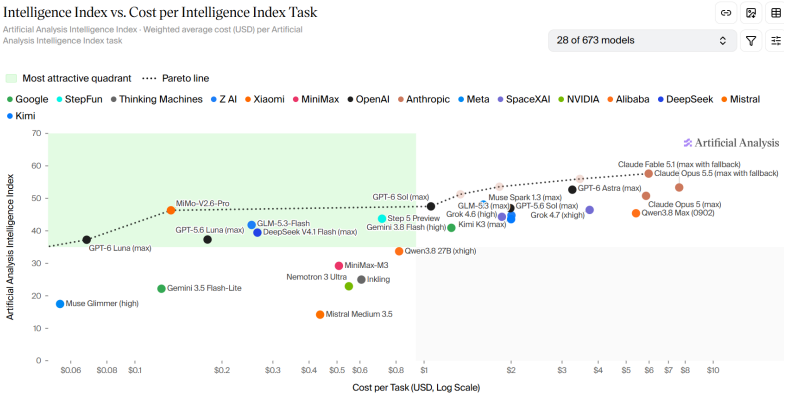
Pareto Optimal
Knee Point
Capability per Dollar
Trade-off



Capability–price scatter and Pareto frontier



Real data: intelligence index vs. cost per task



Horizontal axis: cost per task (log scale, USD); vertical axis: intelligence index. The green upper-left region is the "most watched quadrant"; the dashed line is the Pareto frontier — the further upper-left, the better the value.

Source: Artificial Analysis: Intelligence Index vs. Cost per Intelligence Index Task. Screenshot snapshot 2026-09; original annotation by the institution — cite institution and date.

Reading conventions and data caveats

Every number in this lecture has a shelf life; the method outlives the conclusion.

- Data snapshot: September 2026; sources are credited at the bottom of each page, all from public channels.
- Prices and rankings change frequently; re-verify any conclusion before use.
- Vendor-reported and third-party measured figures are always distinguished.
- Where no reliable source was found, pages say "to be verified" rather than filling in estimates.

Key Terms
Snapshot Date
Vendor-reported
Third-party Measured
Reproducibility

⚠ Tip | A copyable citation template: "According to the *date* leaderboard of *institution*, with configuration *reasoning/tools on or off*, sample size *N*; this conclusion applies to *task type* and not to *boundary*."

02

Who Builds Models: A Five-Layer Stack

Industry layers: from chips to applications

Application layer | office documents, coding assistants, search QA, customer service, image/video creation

Tooling and frameworks | agent orchestration frameworks, Agent SDKs, MCP, vector databases, evaluation and observability

Foundation models | closed-source frontier labs, open-weights camp, cloud-vendor in-house models

Cloud and compute | GPU/TPU clusters, training infrastructure, inference serving and scheduling

Silicon and hardware | NVIDIA, AMD, Google TPU, AWS Trainium, Huawei Ascend, etc.

How to read it: the lower, the more concentrated; the upper, the more fragmented. Investment scarcity concentrates in the bottom three layers; profit imagination in the top two.

US Frontier Labs (I): Four Major Players

OpenAI | founded 2015 ▪ San Francisco

Started as a non-profit research lab; in 2019 created a "capped-profit" subsidiary and took Microsoft investment; flagship products ChatGPT (Nov 2022) and the GPT series; in 2025 completed reorganization: non-profit foundation holds control, operations by a public-benefit corporation (PBC). Openness: mainly closed-source API and subscriptions.

Anthropic | founded 2021 ▪ San Francisco

Founded by former OpenAI research VP Dario Amodei and siblings; safety as its identity, proposed Constitutional AI; flagship Claude series; released the MCP protocol in Nov 2024, now the de facto standard for agent tool interfaces; main investors Google and Amazon.

Google DeepMind | founded 2010 ▪ London

Acquired by Google in 2014; merged with Google Brain in 2023 into Google DeepMind; flagship AlphaGo, AlphaFold, Gemini series; builds its own TPUs — the most complete vertical integration of "model + chip + cloud".

Meta AI (FAIR / MSL) | founded 2013

FAIR, led by Yann LeCun, is a stronghold of deep-learning basic research; since 2023 the Llama series drives the open-weights route, most versions commercially usable under a custom community license; in 2025 formed a superintelligence lab (MSL) and reorganized teams.

US and Europe (II): Differentiated Routes

xAI | founded 2023 ▪ USA

Founded by Elon Musk, deeply integrated with the social platform X; flagship Grok series; built its own Colossus training cluster, competing on "fastest compute expansion"; openness mainly closed, with some historic releases partially open.

Mistral AI | founded 2023 ▪ Paris

Founding team from Meta and DeepMind; Europe's most influential LLM company; known for the dual track of "open weights + commercial API"; flagship Mistral, Mixtral (MoE); seen as the standard-bearer of European "digital sovereignty".

Cohere | founded 2019 ▪ Toronto

Co-founder Aidan Gomez is one of the Transformer paper authors; skips consumer chat and focuses on enterprise retrieval-augmented generation (RAG), embeddings and rerankers; emphasizes private deployment and data residency.

Amazon and Microsoft | cloud-vendor in-house

Amazon launched its Nova series and hosts many third-party models through Bedrock; Microsoft ships the Phi small-model series and its MAI series, and is OpenAI's largest external shareholder and cloud partner. Cloud vendors are "selling water and making water at once".

The China Camp (I): Open Weights and Efficiency

DeepSeek | founded 2023 ▪ Hangzhou

Incubated by the quant fund High-Flyer, founded by Liang Wenfeng; famous for extreme training and inference efficiency; flagship V series (MoE + multi-head latent attention MLA) and R reasoning series; long-standing open-weights policy with major influence on the global open ecosystem.

Alibaba Qwen | since 2023

One of the most thoroughly open model families, covering a full parameter range from 0.5B to ultra-large; most versions under Apache-2.0; among the global leaders in derived fine-tunes, and the practical base of many open-source applications.

ByteDance Seed / Doubao | since 2023

ByteDance's LLM team and product brand, offering APIs through Volcano Engine; aggressive pricing pushed up domestic call volumes; product lines span text, speech, image and video generation.

Moonshot AI | founded 2023 ▪ Beijing

Founded by Yang Zhilin, early differentiated by long context (the Kimi assistant); the later K series uses very large MoE architectures with open weights, focused on agent and tool-calling ability.

The China Camp (II): Scenarios and Industry Side

Zhipu AI (Z.ai) | founded 2019

Grew out of a Tsinghua University knowledge-engineering lab; flagship GLM series; long-running dual track of "open source + commercial", with heavy investment in agents and domestic-chip adaptation.

Baidu ERNIE | since 2023

One of the earliest ChatGPT-style product launches; backed by Baidu AI Cloud and its search ecosystem; emphasizes industry deployment and custom industry models.

Tencent Hunyuan | since 2023

Deployed through Tencent Cloud and the social, gaming and office ecosystems; some model weights released, spanning text, image, video and 3D generation.

MiniMax, iFlytek and Newcomers

MiniMax excels at speech and long context and releases weights; iFlytek Spark builds on its speech-technology strengths; StepFun, 01.AI and others form the second tier, seeking differentiation in multimodality and on-device directions.

The ecosystem layer: not builders, but gatekeepers

Hugging Face | founded 2016 ▪ New York

The world's largest hosting community for models and datasets; open-source libraries such as Transformers are the de facto standard; the "distribution channel" of open-weights models and the main venue for community evaluation and reproduction.

OpenRouter and Other Aggregators

Unify many models behind one API and one bill; route automatically by price, latency or capability; lower comparison and switching costs for users, and add a distribution channel for vendors.

Together AI, Fireworks AI and Other Inference Hosts

Specialize in hosting open-weight models, billed per token, usually cheaper than first-tier labs; also offer fine-tuning, batch inference and private deployment — a key link in open-weights commercialization.

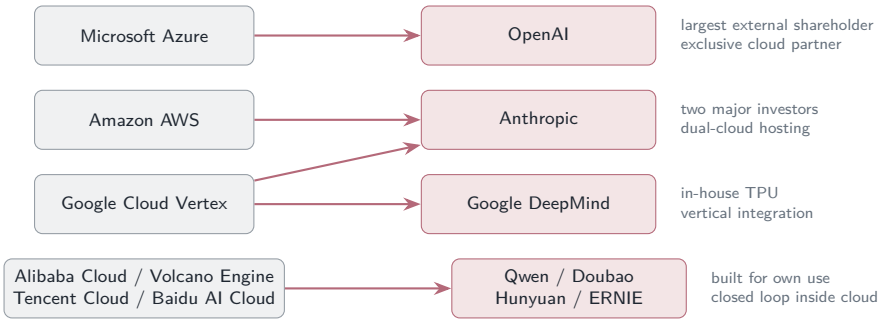
Groq, Cerebras and Other Specialized Inference Hardware

Pursue ultra-low latency and extreme throughput with dedicated chips (LPU / wafer-scale engines); fit latency-sensitive scenarios such as real-time speech and interactive agents; priced differently from general cloud GPUs.

Three routes: closed / open weights / cloud

Dimension	Closed frontier labs	Open-weights camp	Cloud-vendor in-house
Representatives	OpenAI, Anthropic, Google DeepMind	Meta (Llama), Mistral, DeepSeek, Alibaba (Qwen)	Amazon (Nova), Microsoft (MAI), Alibaba, Tencent, Baidu
Revenue	usage-billed API + subscriptions	open weights as funnel + hosting/enterprise services	bundled with cloud resources; models pull compute consumption
Why open	protect technical lead and pricing power	trade for ecosystem, talent and de facto standard	fill model supply on cloud; avoid single-vendor blockage
User benefits	usually the most capable, works out of the box	self-host, fine-tune, no data leaving your control	integrates with existing cloud billing, permissions and compliance
User risks	lock-in, price changes, service shutdown	must build your own engineering and ops	high cross-cloud migration cost; capability often not first-tier

Cloud–model binding: model choice means cloud choice



The same model can differ across clouds in price, rate limits, available regions and data-residency policy

Four Business Models: How Model Companies Earn

Pay-as-you-go

Priced per input/output token; pay for what you use; fits uncertain demand or elastic scaling. Downside: cost grows linearly with usage, and unit prices can change at any time.

Subscription

A fixed monthly fee buys quota and fuller product features (files, code execution, agents); fits individuals and small teams starting fast, but usually not large-scale automated calls, and terms of use often restrict it.

Enterprise Agreements and Private Deployment

Per-seat or annual contracts with data isolation, dedicated capacity, compliance commitments and support; terms are negotiable — price, rate limits, data clauses; common in finance, healthcare and government.

Open Weights + Hosted Services

Weights download free; revenue comes from hosted inference, fine-tuning, support and enterprise licenses; users gain control and bargaining power at the cost of building engineering, ops and security themselves.

Open Weights Are Not "Free to Use however You Like"

Weights can be downloaded, but license terms differ sharply — check every clause.

- Permissive licenses (Apache-2.0, MIT): commercial use, modification and redistribution, fewest restrictions.
- Custom community licenses: often carry monthly-active-user thresholds, naming requirements, or bans on training competing models.
- Some "open source" releases open weights only, not data or code — limited reproducibility.
- Before use, confirm: commercial use allowed? renaming allowed? attribution required? scale thresholds?

Key Terms

Open Weights
Open-source License
Commercial Use
Redistribution

Example | Permissive licenses (Apache-2.0, MIT) allow free commercial use and redistribution; custom community licenses often carry user thresholds, naming requirements or "no training competing models" clauses. Both are called "open" — the rights differ completely.

Company background master table

Organization	Founded	Key capital/binding	Openness	Flagship family	Selection focus
OpenAI	2015	Microsoft, funds	closed	GPT series	capability frontier, mature ecosystem, lock-in risk
Anthropic	2021	Google, Amazon	closed	Claude series	long context and agents, enterprise compliance
Google DeepMind	2010	wholly Alphabet	closed + partial open	Gemini series	in-house chips, multimodal, ultra-long context
Meta AI	2013	wholly Meta	open weights	Llama series	self-hosting, license varies by version
xAI	2023	own funds + raises	mostly closed	Grok series	aggressive compute scaling, young ecosystem
Mistral AI	2023	European capital & industry	open weights + API	Mistral, Mixtral	European data sovereignty, MoE efficiency
Cohere	2019	enterprise and cloud capital	closed + partial open	Command, Embed	enterprise RAG and embeddings
DeepSeek	2023	High-Flyer incubation	open weights	V series, R series	high value, self-hosting
Alibaba	—	Alibaba Cloud	mostly open weights	Qwen	full parameter range, Apache-2.0
ByteDance	—	Volcano Engine	closed + partial open	Doubao	low price, speech and video generation
Moonshot AI	2023	venture capital	partial open weights	Kimi K series	long context, agent tool calling
Zhipu AI	2019	state and industry capital	dual track	GLM series	domestic-chip adaptation, agents

Source: Summarized from public vendor information; confirm founding dates and capital structure from official disclosures. This table contains no unverified funding or valuation figures.

Why company background affects selection

Beyond technical metrics, supply continuity and terms are the long-run cost.

- Capital structure determines route stability: a company dependent on one investor may change openness overnight.
- Cloud binding determines migration cost: deep model–cloud coupling means "changing models" often means "changing clouds".
- Openness determines bargaining power: downloadable weights are your fallback when a service shuts down.
- Regulation and compliance: litigation, export controls and data law in different jurisdictions affect availability.

Key Terms

Vendor Lock-in
Supply Continuity
Bargaining Power
Compliance Risk

Formula | Switching cost $C_{sw} = C_{prompt} + C_{eval} + C_{tool} + C_{compliance}$: prompt rewriting, evaluation rebuilding, tool adaptation and compliance re-review all count. It often exceeds the unit-price gap between models.

Three Judgments to Remember from This Section

Judgment about the industry landscape matters more than memorizing version numbers.

- The most capable models are usually the least open — openness is a business strategy, not a technical stage.
- Chinese vendors contribute significantly on open weights and price-performance; the global open ecosystem is now interdependent.
- The ecosystem layer (hosting, aggregation, tools) is becoming a new, non-ignorable bargaining link.

Key Terms

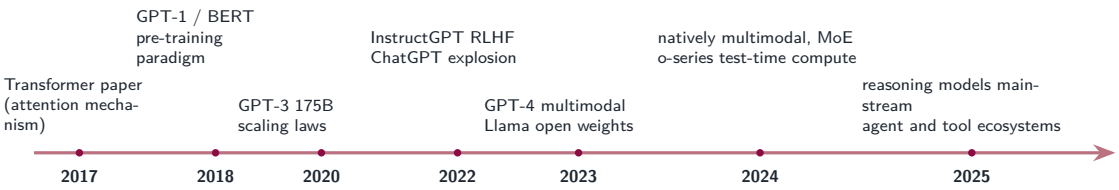
Open-weights Camp
Vertical Integration
Distribution Channel
De Facto Standard

Diagram | Order of judgment: openness first (can you self-host?) → capital binding (is the route stable?) → cloud dependence (migration cost) → jurisdiction and compliance (can you use it at all?).

03

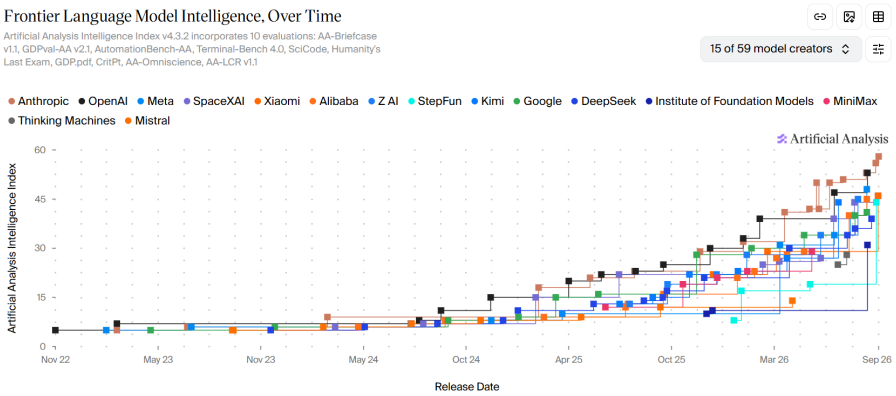
From Transformer to Reasoning Models: Three Threads

Timeline 2017–2025: attention to reasoning models



Note: this section lists only public, verifiable milestones; the 2026 version landscape must be updated from official releases.

Real data: frontier-model intelligence index over time



Under one institution's methodology, the frontier intelligence index rises year by year; where the curve flattens is exactly where "equal gains need more cost".

Source: Artificial Analysis: Frontier Language Model Intelligence, Over Time. Screenshot snapshot 2026-09; original annotation by the institution — cite institution and date.

Stage One: Attention Unifies the Architecture

Transformer replaced recurrence with attention, enabling massive parallel training.

- In 2017 "Attention Is All You Need" introduced the Transformer architecture.
- In 2018 two branches appeared: GPT went "decoder + generation"; BERT went "encoder + understanding".
- The key change: training parallelizes — model size is no longer capped by sequential computation.
- Today the vast majority of LLMs still build on this architecture.

Key Terms

Attention
Self-attention
Parallel Training
Encoder/Decoder

Formula | Attention: $\mathrm{softmax}\left(QK^T/\sqrt{d_k}\right)V$. Every token can "see" every other token directly, instead of passing information step by step — the precondition of parallel training.

Stage two: scaling laws and pre-training

As parameters, data and compute grow together, capability improves predictably.

- In 2020 GPT-3 (175B parameters) proved "few-shot prompting" works.
- Scaling laws turned budget allocation from experience into an estimable engineering problem.
- The consensus of this stage: scale general capability first, then handle specific tasks.
- Limits appeared too — training is extremely expensive, and the model does not automatically become "obedient" or "safe".

Key Terms

Scaling Law
Pre-training
Few-shot Learning
Emergent Ability

Formula | Scaling law: loss $L(N, D)$ falls roughly as a power law in parameters N and data D. It turned "should we invest more" from intuition into an estimable engineering question.

Stage three: alignment, obedience and safety

Alignment addresses "capable but not acting as humans intend".

- Instruction tuning: teach the model to complete tasks from "instruction — ideal answer" pairs.
- RLHF: train a reward model from human preference rankings, then optimize the policy.
- In 2022 InstructGPT and ChatGPT proved: alignment improves experience more directly than raw scale.
- Side effects appeared: sycophancy, over-refusal, and "sounding more confident".

Key Terms

Instruction Tuning
RLHF
Preference Alignment
Sycophancy

Example | The same question: "I don't know" vs. "make up a plausible answer" — the former is not naturally favored by the training objective. That is exactly why alignment is necessary.

Stage Four: From “Bigger” to “Thinking Longer”

Test-time compute shifts the race from training scale to inference-time allocation.

- Reasoning models run longer internal computation before answering.
- Core idea: trade inference-time compute for accuracy on hard tasks — math, code, planning.
- The price is latency and cost — reasoning tokens are billed too.
- A new tunable parameter appears: the “thinking budget” — harder tasks get more budget.

Key Terms

Reasoning Model
Test-time Compute
Chain-of-Thought
Thinking Budget

Formula | Total cost $\approx$ thinking tokens $\times$ output price. Doubling the thinking budget roughly doubles cost while accuracy gains are usually a few points — diminishing returns are the norm for reasoning models.

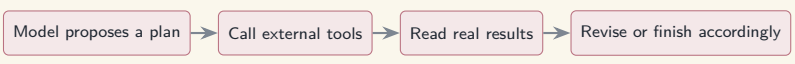
Stage Five: From Generating Text to Using Tools

Models acquired the interface ability to "reach the outside world".

- Tool calling lets the model emit structured calls, not just natural language.
- Agents compose "model + context + tools" into a loop that completes tasks.
- Coding, retrieval, browser and computer operation matured first.
- The core competence of this stage shifts from model parameters to harness engineering.

Key Terms

Tool Calling
Agent
Harness Engineering
Function Calling



Five paradigm stages and their bottlenecks

Stage	Milestones	Core bottleneck solved	Left open
Architecture unified	Transformer (2017)	sequence modeling could not parallelize	needs massive data and compute
Scale	GPT-3 (2020) and scaling laws	task-specific models did not generalize	ignores human intent; costly
Alignment	InstructGPT / ChatGPT (2022)	capable but disobedient, unsafe	sycophancy, hallucination, over-refusal
Inference compute	o-series reasoning models	multi-step reasoning accuracy too low	latency and cost rise sharply
Tools and agents	tool calling, MCP, coding agents	text-only output, no ability to act	permissions, reliability, evaluation hard

An Independent Line: Chinese LLMs

Chinese vendors took a different road on open weights and engineering efficiency.

- Early entries led with chat products; then came the dense-release "battle of a hundred models".
- Openness became a key strategy: Qwen, GLM, DeepSeek, Kimi and others released weights.
- An efficiency route stands out: under compute constraints, engineering (sparsity, quantization, inference scheduling) weighs more.
- Result: in the global open-source community, Chinese models became the practical base of many derivative models.

Key Terms

Open Weights
Distillation
Quantization
Derivative Model

Example | Idioms, officialese register, Chinese–English mixing and punctuation conventions are where international and domestic models differ most; evaluate per target language, not by total score alone.

Architecture Evolution: from Dense to Mixture-of-Experts

Under the same compute budget, sparsity makes "large and fast" possible.

- Dense models: every token activates all parameters — simple, but inference cost grows linearly with scale.
- Mixture-of-Experts (MoE): each token activates only a few experts — huge total parameters, small per-call compute.
- The two numbers that matter are "total parameters / active parameters" — never read only the total.
- The price: high VRAM use, complex routing training; deployment difficulty does not fall automatically.

Key Terms

Dense Model
Mixture-of-Experts, MoE
Active Parameters
Routing

Formula | MoE: active parameters $\ll$ total parameters. One publicly disclosed configuration is "about 671B total / about 37B active" — large capacity, small per-call compute (figures per official technical reports).

The Context-Window Arms Race

From 4K to million-token windows: "how much it can see" and "how well it can use it" are two things.

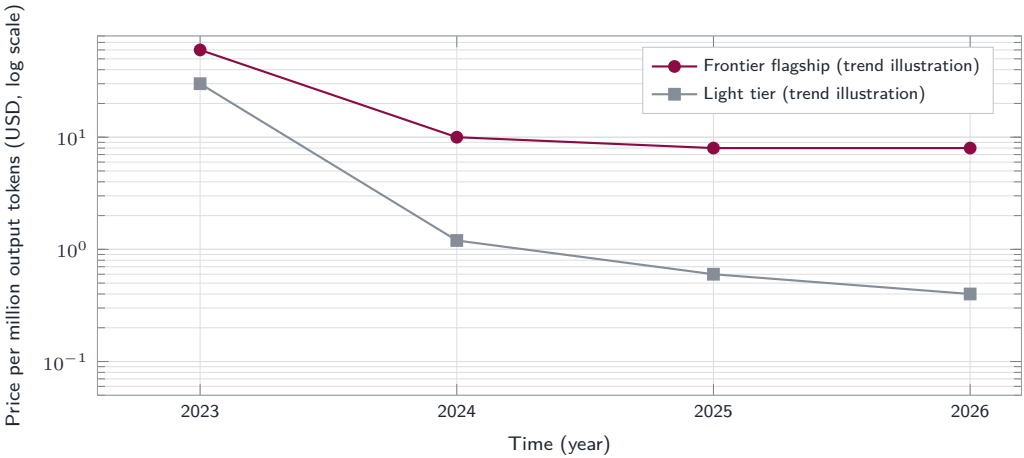
- Early models held only a few thousand tokens; long documents had to be split.
- Long-context techniques (position extrapolation, attention approximations) pushed windows up by orders of magnitude.
- A bigger window is not better use: mid-window information is easily ignored ("lost in the middle").
- Engineering conclusion: ultra-long input still needs splitting, retrieval and structured summaries.

Key Terms

Context Window
Positional Encoding
Lost in the Middle
RAG

Example | Lost in the middle: put a key clause in the middle of a very long input and the model more easily misses it. Splitting, re-ranking and structured summaries beat simply extending the window.

Price history: cost at equal capability falls fast



Three Threads to Remember from This Section

Scale, alignment, test-time compute — nearly all progress fits one of them.

- Scale thread: parallel architecture + ample data and compute ⇒ predictable general gains.
- Alignment thread: capability must match human intent, or "smarter" becomes more dangerous.
- Compute thread: moving compute from training to inference is the fastest recent gain.
- Beyond the three, falling cost is what actually made LLMs ubiquitous.

Key Terms
Paradigm
Predictable Scaling
Inference Cost
Capability Commoditization

Diagram | Historical thread: parallel architecture → scale dividend → alignment → test-time compute → tool use. Each step solved the bottleneck exposed by the last.

04

From Talking to Working: Thirty Years of Agents

The Minimal Agent Formula

Agent = model + context + tools; none of the three is optional.

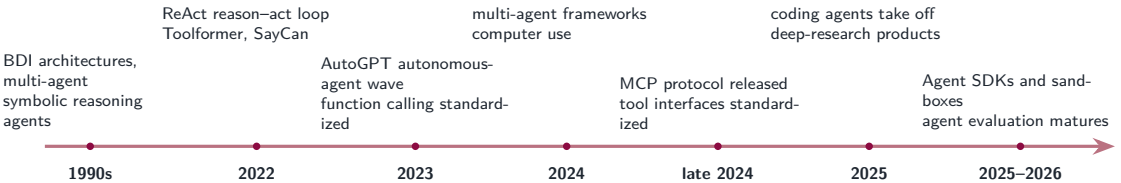
- The model understands goals, plans steps, chooses the next action.
- The context carries task, history, knowledge, observations and current state.
- The tools provide real-world read/write: search, computation, files, databases, business APIs.
- Model without tools is a chatbot; tools without a model are classic automation scripts.

Key Terms

Agent
Context
Tool Calling
Loop

Formula | Agent = LLM + Context + Tools. Missing any one stalls the task: without context it does not know the situation; without tools it cannot change the world.

Agent timeline: symbolic reasoning to tool use



Prehistory: Agents Are Not a New Idea

"Acting autonomously" has been studied for thirty years — only the "brain" changed.

- 1990s multi-agent systems studied negotiation, cooperation and competition.
- The BDI architecture described agents by Belief, Desire and Intention.
- Classic RL agents learned policies in games and robotics, but choked on open-ended language tasks.
- The shared bottleneck never changed: representing open-world knowledge as computable decisions.

Key Terms

Multi-agent System
Belief-Desire-Intention
Reinforcement Learning
Symbolic Reasoning

Example | BDI's "belief–desire–intention" and today's "state–goal–plan" are the same idea in different clothes; only the "reasoning engine" changed, from logic rules to a language model.

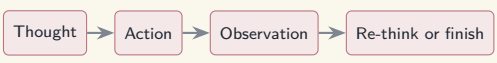
2022: ReAct, Alternating Reasoning and Action

ReAct braided "thinking" and "doing" into one loop — the prototype of the modern agent.

- The core form repeats "Thought – Action – Observation".
- Reasoning decides what to check next; action calls search or compute; observation writes real results back into context.
- Versus reasoning alone, ReAct lets the model correct its own errors with outside information.
- Limits showed too: loops can idle, and failures invite endless retries.

Key Terms

ReAct
Observation
Tool Use
Looping



2023: The AutoGPT Wave and Function Calling

The same year delivered both the "excess autonomy" lesson and the "standard interface" answer.

- Autonomous-agent projects (AutoGPT, BabyAGI, etc.) let the model decompose and run long tasks, drawing wide attention.
- Field tests exposed the problems: loops, runaway cost, error accumulation, unverifiable results.
- The same year, function calling appeared: the model emits structured requests, the program executes.
- Consensus settled: build reliable interfaces and boundaries first; discuss autonomy later.

Key Terms

Autonomous Agent
Function Calling
Structured Output
Runaway Cost

Example | The classic autonomous-agent failure: with no turn or budget cap, it retries the same failed action forever, burning budget in useless loops — the shared lesson of the first wave.

2024: Multi-Agent and Computer Use

From "calling functions" to "operating whole software interfaces".

- Multi-agent frameworks (AutoGen, CrewAI, etc.) orchestrate cooperation via roles and message passing.
- Computer use lets the model see the screen, move the mouse and type.
- Web agents turn page operations into programmable tasks.
- The gain is coverage; the price is more failure paths and sharper verification and permission issues.

Key Terms

Multi-agent Orchestration
Computer Use
Web Agent
Role Assignment

Example | "Book me the cheapest flight" needs reading mail, opening a browser, comparing prices, ordering — any failed step breaks the chain; computer-use agents therefore need retry policies and human takeover.

Late 2024: MCP Standardizes Tool Interfaces

Tool integration moved from "every vendor writes their own" to "implement the protocol once".

- The Model Context Protocol (MCP) was proposed and open-sourced by Anthropic in November 2024.
- Goal: unify how models discover and call external tools and data sources.
- Its value is like USB-C: a tool implements once, and many models and clients reuse it.
- Adopted widely since, a de facto standard; security boundaries (permissions, injection) became the focus.

Key Terms

MCP
Server/Client
De Facto Standard
Prompt Injection

Formula | Integration cost falls from $N \times M$ to $N + M$: N models each wiring M tools is a product; with one protocol it becomes a sum. That is the economic value of the protocol layer.

2025: Coding Agents and Deep Research

The highest-value scenarios ran first, because they have objective acceptance criteria.

- Coding agents read repositories, edit code, run tests, submit changes.
- Why they ran first: compilers, tests and version control give objective feedback — automatically verifiable.
- Deep-research products orchestrate multi-round retrieval, reading and synthesis into one task.
- The common point: the goal is "complete the task", not "produce a passage".

Key Terms

Coding Agent
Verifiable Feedback
Deep Research
Task Success Rate

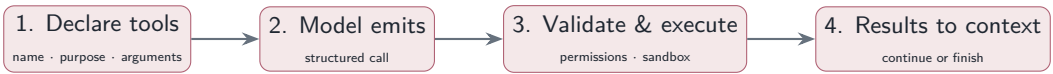
Example | Coding agents ran first because compilers and test suites provide an objective right/wrong signal: the agent can tell by itself whether it succeeded — verifiability is the premise of all automation.

Protocol layers: MCP and interoperability

Layer	Representative	Problem solved	Analogy
Model ↔ tools	MCP (proposed 2024.11)	unify discovery, description and invocation of tools and data sources	universal connector for devices and accessories
Agent ↔ agent	A2A and peers	how agents from different vendors delegate and cooperate	inter-company handoff processes
Application ↔ model	vendor API specs	unified calling format for prompts, tools, streaming	different power plugs
Orchestration	LangChain, AutoGen, CrewAI, etc.	express multi-step, multi-role flows as maintainable programs	project-management templates

Protocol competition is ongoing; for versions and adoption, defer to the official specifications.

The Four-Step Tool-Calling Loop



Failure can also be returned: the model must distinguish “call failed” from “empty result”

Context Engineering: More Important than Prompts

The prompt is only a small part of the context.

- Context engineering arranges: instructions, examples, retrieved evidence, tool results, history, state.
- Memory splits into short-term (current task) and long-term (cross-session preferences and facts).
- The longer the context, the more noise, and the easier mid-window information is ignored.
- Common moves: summarizing compaction, structured state tables, retrieve-then-inject, dropping stale observations.

Key Terms

Context Engineering
Memory
State Table
Compaction

Diagram | What context contains: system instructions + current task + retrieved evidence + tool returns + conversation history + structured state. Context engineering decides the inclusion and ordering of these six.

Sandboxes and permissions before deployment

How much authority to grant depends on how fast you can detect and undo errors.

- A sandbox confines execution to an isolated environment: filesystem, network and processes all constrained.
- Least privilege: grant only the capabilities the task needs.
- Irreversible actions (payment, sending, deletion, publishing) require human approval or dual review.
- Key designs: maximum turns, budget caps, failure thresholds, rollback plans.

Key Terms

Sandbox
Least Privilege
Human Approval
Circuit Breaker

Example | The irreversible-action list: payment, outbound sending, data deletion, public release, making commitments. These must be human-approved — once the error happens, retrying cannot undo it.

The six-level agent capability ladder

Level	Form	Capability	Typical risk
L1	conversational QA	single- or multi-turn text interaction, no external data	hallucination, stale information
L2	retrieval-augmented	answers grounded in a knowledge base, can cite sources	retrieval misses, wrong citations
L3	tool calling	can call search, computation, APIs to fetch and compute	wrong arguments, unauthorized calls
L4	workflow	fixed-path multi-step automation, model inside nodes	uncovered exception branches
L5	autonomous agent	path decided dynamically by the model, can loop and retry	runaway cost, error accumulation
L6	multi-agent	role division, mutual checking and delegation	unclear accountability, distortion in handoff

Higher on the ladder is not automatically better: a task satisfiable at L1/L2 should not run at L5.

Why Agent Evaluation Is Especially Hard

Multi-step success rates amplify tiny single-step errors into disasters.

- A model with 95% single-step accuracy succeeds at $0.95^{20} \approx 36\%$ over 20 steps.
- The environment has state: re-running the same task may differ; reproducibility is hard.
- Scoring is hard: right answer, flawed process; right process, failed result.
- So evaluation must track pass rate, cost, step count and violations together.

Key Terms

Error Compounding
Reproducibility
Verifiable Environment
Step Budget

Formula | Error compounding: 95% per-step accuracy gives $0.95^{20} \approx 36\%$ over 20 steps. The more steps, the harsher the demand on per-step reliability.

05

Classify Before You Choose: Seven Dimensions

Why Classify First

Classification is not an academic game — it collapses "thousands of models" into "a few candidates".

- Models multiply faster than human memory; comparing one by one is impossible.
- Classification dimensions map to real constraints: modality, capability, deployment, license, cost.
- Narrow the candidate set by dimension first, then compare performance inside it — the most efficient order.
- At each step ask: "is this constraint hard, or negotiable?"

Key Terms

Taxonomy
Candidate Set
Hard Constraint
Narrowing

Example | "Recognize product photos" is multimodal, "write a weekly report" is text, "build vector search" is embedding — classify wrong and every later comparison is void.

Dimension 1: Classification by Modality

Modality	Capability	Typical uses and directions
Text-only	understand and generate text	summarization, QA, extraction, writing, code
Multimodal understanding	read text plus image/audio/video	document recognition, chart understanding, scene description, screen operation
Image generation	text-to-image, image-to-image, editing	marketing material, product images, design sketches
Video generation	text-to-video, image-to-video	short-video material, animated demos
Speech	recognition, synthesis, real-time dialogue	meeting notes, voice assistants, dubbing
Embedding and reranking	map text to vectors and order them	semantic search, recommendation, clustering, dedup
World models and action	predict environment state, output actions	robotics, autonomous driving, game agents

A "multimodal model" usually reaches usable level in only some modalities — evaluate each direction separately.

Dimension 2: By Capability Positioning

Things called "LLM" may have completely different design goals.

- General chat models: balanced, fast-responding, fit daily interaction and content generation.
- Reasoning models: longer internal computation before answering, fit math, code and complex planning.
- Code models: optimized for programming, deeply integrated with repos, terminals and test environments.
- Retrieval-QA and embedding models: serve knowledge-base retrieval; they cap what RAG can achieve.

Key Terms

General-purpose
Reasoning Model
Code Model
Embedding Model

Example | Three things all called "LLM": a chat-product base (balanced, fast), a contest-grade solver (slow, pricey, accurate), a code-completion kernel (specialized, cheap). Their optimization targets differ completely.

Dimension 3: By Architecture and Efficiency

Look at two numbers: total parameters and active parameters.

- Dense: every token activates all parameters; predictable behavior, VRAM grows with scale.
- MoE: only part of the experts activate; huge total parameters, relatively small per-call compute.
- Example: "671B total / 37B active" means large capacity, small per-call cost.
- Sparsity does not lower the VRAM bar: all expert weights must be loaded; deployment stays expensive.

Key Terms

Dense
MoE
Active Parameters
VRAM Footprint

Formula | VRAM $\approx$ parameter count $\times$ bytes per parameter (FP16 about 2 bytes, 4-bit about 0.5) + activations and cache. VRAM often becomes the bottleneck before compute does.

Dimension 4: By Openness

Openness determines your bargaining power and your fallback.

- Closed API: service only, no weights; usually strongest capability, weakest control.
- Open weights: downloadable, self-hostable, fine-tunable, but license terms vary widely.
- Open source (code and data): reproducible and auditable, but fully open frontier models are rare.
- On-premise deployment: runs in your own environment for data residency, but requires ops capability.

Key Terms

Closed-source
Open Weights
Self-hosting
On-premise Deployment

Example | "Data must not leave our control" forces open weights and self-hosting; the most frontier capability usually only comes via closed APIs. The two roads rarely coexist — you must rank them.

Dimension 5: By Scale and Deployment Position

The same task trades off completely differently between a small on-device model and a large cloud model.

- Frontier models: strongest, must be called online, highest unit cost.
- Mid-size models: the balance point of capability and cost, fit high-volume processing.
- Small models (SLM): run on one card or even on-device, fit narrow tasks — classification, extraction, rewriting.
- On-device models: data never leaves the device, best privacy, but capability and context clearly limited.

Key Terms

Small Language Model
On-device Inference
Cloud-Edge
Distillation

Example | A 3B-class on-device model handles classification, extraction and rewriting fine, but is clearly insufficient for multi-step reasoning and long-document understanding — "it runs" and "it can be trusted" are two things.

Dimensions 6 and 7: Interaction Form and Price Tier

Form decides how you use it; tier decides what you pay.

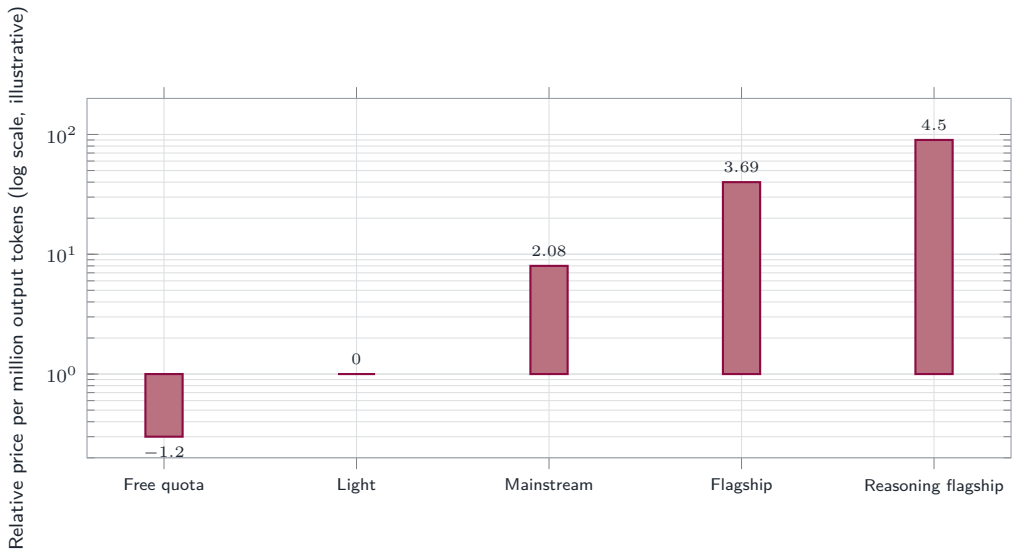
- Interaction form: chat interface, API, copilot, agentic, multi-agent.
- Price tiers: flagship, mainstream, light, free quota — four tiers, four different usage strategies.
- The same vendor often deploys across several tiers, price-discriminating on a "capability ladder".
- Classic mistakes: using a flagship for classification, or a light model for complex reasoning.

Key Terms

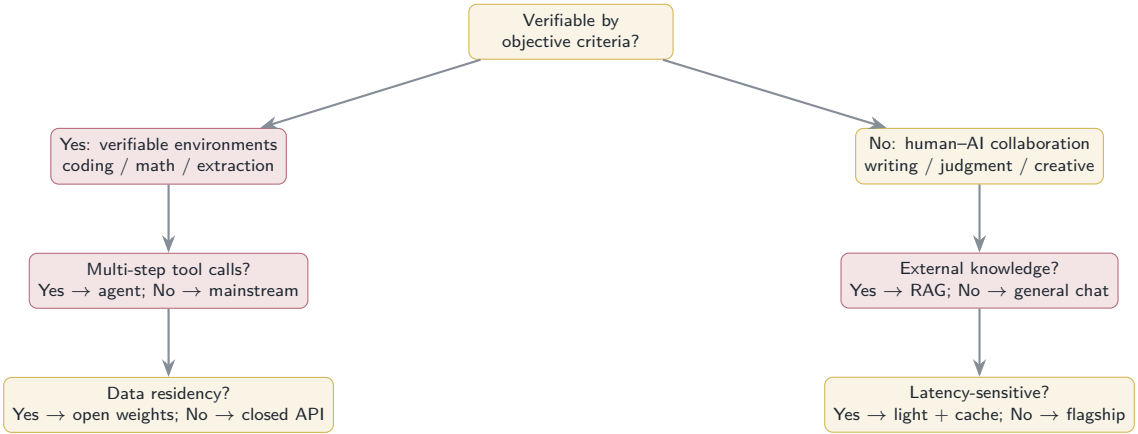
Copilot
Agentic
Price Discrimination
Capability Ladder

Diagram | Four price tiers, four strategies: free quota for experiments, light for batch, mainstream for production workhorses, flagship and reasoning only for high-value hard problems.

Relative unit price of four tiers (illustrative)



Classification Decision Tree: from Task Back to Model Class



Classification matrix: seven dimensions

Dimension	Common values	Direct meaning for selection
Modality	text / multimodal / image / video / speech / embedding	decides whether your input and output forms are supported; prompts cannot compensate
Capability	general / reasoning / code / retrieval	decides whether you pay a premium for "thinking longer"
Architecture	dense / MoE	decides VRAM needs, deployment mode and throughput
Openness	closed / open weights / open source / on-premise	decides bargaining power, compliance path, shutdown fallback
Scale and position	frontier / mid / small / on-device	decides unit cost and privacy boundary
Interaction form	chat / API / copilot / agentic	decides integration workload and verification mode
Price tier	free / light / mainstream / flagship	decides usage strategy: batch or not, experiment or not

How to use: narrow top-down, one dimension at a time, keeping only viable options; usually three rounds shrink the candidate set to 3–5.

Three Common Classification Mistakes

Classify wrong, and however precise the later comparison, it is meaningless.

- Mistake 1: reading "more parameters" as "more capable" — after sparsification, total parameters and effective capability are not linearly related.
- Mistake 2: reading "open source" as "free" — self-hosted GPUs, power and labor must be counted.
- Mistake 3: reading "multimodal" as "omnipotent" — most models perform well in only 1–2 modalities.
- Correct order: derive the class from the task's needs, not the task from a model's label.

Key Terms

Parameter Fallacy
Hidden Cost
Capability Boundary
Task-driven Selection

Example | Reverse the three mistakes: what capability does this task truly need? Which class can supply it? Among qualified candidates, which is the best value? — run the order backwards and the conclusion usually changes completely.

06

Who Evaluates: Institutions, Mechanisms, and Reading Results

Why Third-Party Evaluation Is Necessary

Between vendor-reported scores and real capability stands a layer of self-interest.

- Vendor benchmarks allow selective reporting: only favorable benchmarks and configurations.
- With opaque conditions, the same model can be "tuned" into very different scores.
- The value of third parties: unified question sets, unified configurations, cross-checkable.
- But third parties have stances too: who funds them, do they sell evaluation services, is raw data published?

Key Terms

Vendor Benchmark
Conflict of Interest
Cross-validation
Methodological Transparency

⚠ Tip | On seeing "No. 1 on benchmark X", ask four things: who ran it? is the test set public? under what configuration? what sample size and confidence interval?

The evaluation landscape: seven types

Type	Representatives	Coverage	Scoring	Question answered
Arena	LMArena, AA arenas	pairwise battles judged by humans	Elo / Bradley–Terry	which answer do people prefer?
Composite index	Artificial Analysis, SEAL	weighted benchmarks + speed and price	weighted index / sub-boards	overall capability and value?
Academic benchmark	HELM, LiveBench, Epoch AI	standardized sets, trend tracking	accuracy / efficiency	where is the academic capability boundary?
Domain-specific	Vals AI, MedHELM	law, finance, medicine	in-domain accuracy	usable in professional scenarios?
Agent-specific	SWE-bench, Terminal-Bench, τ^2-bench, GAIA	multi-step tasks and tool use	task pass rate	can it actually finish the job?
Chinese and regional	OpenCompass, SuperCLUE, FlagEval	Chinese context and domestic models	composite / radar	how good is Chinese performance really?
Enterprise service	Galileo, Patronus, Arize, RAGAS	hallucination, safety, observability	custom metrics	how to monitor after launch?

Source: The typology is a teaching simplification; one institution often spans several types (e.g. Artificial Analysis offers both an index and arenas).

LMArena: Pioneer of Human-Preference Battles

Background

Originated from the LMSYS project at UC Berkeley; the first to turn "anonymous pairwise battles + human votes" into a public leaderboard; now independently run, with text, vision, image and video arenas.

Mechanism

Models are paired anonymously and users vote for the better answer; the Bradley–Terry model fits win–loss relations into Elo scores; a style-control variant weakens the "longer, prettier answer wins" bias; per-domain boards and confidence intervals are available.

How to Read

Official entrance: `https://lmarena.ai/`. Check three things: (1) is style control on; (2) do confidence intervals overlap adjacent ranks; (3) do the domain boards (coding, math, creative writing, hard prompts) match your focus. This lecture uses no unverified specific scores.

Pros & Limits

Pros: real user preferences, large samples, continuously updated, with confidence intervals. Limits: measures "likability" more than "correctness"; affected by voter demographics; long tasks and tool use are not reflected.

Artificial Analysis: independent measurement

Background

An independent analysis firm that calls vendor APIs directly and does not rely on vendor-reported numbers; measures intelligence, output speed, first-token latency and price together, and runs image, video and speech arenas.

Mechanism

The composite Intelligence Index weights many public benchmarks — knowledge and reasoning, math, science, coding, long context, instruction following, agent tasks; speed metrics are output tokens per second and first-token latency; prices are normalized on its own billing basis into "cost per thousand tasks".

How to Read

Official entrance: `https://artificialanalysis.ai/`. Points: (1) the index has version numbers — never compare across versions directly; (2) check whether reasoning mode is on — reasoning models differ hugely in cost and latency; (3) the site has a "value" view showing capability per dollar. This lecture uses no unverified specific scores.

Pros & Limits

Pros: measured speed and price — a rare "three dimensions at once"; charts usable for decisions directly. Limits: the weighting is set by the institution; changing weights can change the winner; some benchmark sets are public, inviting targeted optimization.

Stanford HELM: academic transparency

Background

Produced by Stanford CRFM; full name Holistic Evaluation of Language Models; the representative of "all scenarios, many metrics, reproducible" evaluation.

Mechanism

For every scenario it reports many metrics (accuracy, calibration, robustness, fairness, bias, efficiency), emphasizing standardized prompts and reproducibility; derivative series cover capabilities, safety, long context (HELMET), medicine (MedHELM), vision-language (VHELM) and more.

How to Read

Official entrance: `https://crfm.stanford.edu/helm/`. Points: (1) HELM's value is decomposed dimensions, not a single ranking; (2) use it to answer "in which abilities does this model have systematic weaknesses"; (3) question sets change between versions — check release dates. This lecture uses no unverified specific scores.

Pros & Limits

Pros: open methodology, diverse metrics, strong academic neutrality; fits teaching and audit. Limits: slower update cadence than commercial boards; academic question sets sit far from product experience; scores do not translate directly to business value.

Epoch AI: Capability Trends and Expert-Level Math

Background

Run by the research institute Epoch AI, focused on long-run data-driven trends of AI capability and compute; its "benchmarking hub" aggregates several boards and offers its own very hard math benchmark.

Mechanism

Key features: (1) expert-written, never-public private question sets measure frontier math with less contamination; (2) a Capability Index maps many benchmarks onto one scale for cross-time comparison; (3) input-side trends — training compute, data, model size — are tracked in parallel.

How to Read

Official entrance: `https://epoch.ai/`. Points: (1) watch trend lines, not single points; (2) private sets make it insensitive to benchmark gaming — good for observing the true frontier; (3) difficulty is far above everyday tasks — do not extrapolate to ordinary business. This lecture uses no unverified specific scores.

Pros & Limits

Pros: contamination-resistant, long-run comparable, covers both inputs and outputs. Limits: difficulty so high that discrimination crowds the tail; questions not public, details hard to verify externally; research-paced updates.

LiveBench: Designed Against Contamination

Background

A benchmark maintained by researchers from several universities and institutes, with "avoiding data contamination" as the first design principle; questions roll over monthly.

Mechanism

Questions keep refreshing from recent news, latest papers, newly released datasets and contest problems, ensuring they postdate model training cutoffs; covers math, reasoning, coding, data analysis, language understanding and instruction following, and requires objectively gradable answers.

How to Read

Official entrance: `https://livebench.ai/`. Points: (1) monthly updates — cross-month comparison needs checking whether questions changed; (2) category scores carry more information than the total, especially coding and reasoning; (3) subjective tasks (writing) are excluded — look elsewhere for those. This lecture uses no unverified specific scores.

Pros & Limits

Pros: clear anti-contamination design, frequent updates, objectively gradable. Limits: questions tend to be short and objective, missing long tasks; high-frequency updates make historical trend lines awkward to read.

ARC Prize and ARC-AGI: abstract reasoning

Background

Driven by the ARC Prize foundation around the Abstraction and Reasoning Corpus, aiming to measure generalization to entirely new rules.

Mechanism

The task: infer the transformation rule of visual grids from few examples and output results; the test set stays secret against contamination; new versions add "efficiency scoring" — the cost per problem counts, rewarding solving with less compute.

How to Read

Official entrance: `https://arcprize.org/`. Points: (1) easy for humans, hard for models — a probe of generalization; (2) efficiency scoring ends the era when "more compute" was unconditionally optimal; (3) low scores do not mean useless — they signal reliance on pattern memory. This lecture uses no unverified specific scores.

Pros & Limits

Pros: low contamination risk, aimed at the essence of generalization, adds a cost dimension. Limits: an unusual task form with limited relation to language and business tasks; pass rates long low, discrimination concentrated among few models.

SEAL (Scale AI): Private, Expert-Written Evaluation

Background

Run by the data and evaluation company Scale AI, built on "expert-written, privately held" evaluation sets covering coding, agents, reasoning, math, multilingual, instruction following, long context and safety.

Mechanism

Each sub-board is authored and kept private by domain experts, avoiding training-data leakage; results publish as leaderboards with confidence intervals, so rank gaps can be judged for significance.

How to Read

Official entrance: `https://scale.com/leaderboard`. Points: (1) check confidence-interval overlap first — overlap means the rank gap is not significant; (2) sub-boards are more trustworthy than totals; (3) questions are not public, so you cannot re-verify — you trust the expert process. This lecture uses no unverified specific scores.

Pros & Limits

Pros: private sets resist contamination, fine-grained domains, statistical uncertainty reported. Limits: corporate operation, limited methodological disclosure; coverage varies with expert capacity; no independent reproduction.

Vals AI: "Can It Do the Job" in the Professions

Background

A firm specializing in professional-domain evaluation, building benchmarks around law, finance, medicine and tax, with questions written and graded by licensed professionals.

Mechanism

Designed around "professional tasks": legal document review, financial modeling, accounting treatment, medical QA; some sub-boards use expert grading or comparison against standard answers, stressing task completion over fluency.

How to Read

Official entrance: `https://www.vals.ai/`. Points: (1) this is domain-fit evaluation — general flagships do not automatically lead; (2) check whether the evaluated tasks match your industry — do not extrapolate across domains; (3) ask whether grading is done by professionals. This lecture uses no unverified specific scores.

Pros & Limits

Pros: close to real work products, high-quality professional grading. Limits: narrow domains, updates bounded by expert capacity; difficulty not comparable across sub-boards; questions and grading not fully public.

OpenCompass: Chinese comprehensive evaluation

Background

An open-source evaluation system led by the Shanghai AI Laboratory, aimed at Chinese and multilingual scenarios; among the most influential comprehensive platforms in China.

Mechanism

Organizes evaluation sets along dimensions such as knowledge, reasoning, language, long text, safety and agents; offers the CompassRank composite board and sub-boards; the toolchain is open, letting institutions rebuild and reproduce evaluations.

How to Read

Official entrance: `https://opencompass.org.cn/`. Points: (1) read the composite and sub-boards together; (2) good coverage of Chinese and domestic models — fit for first-pass Chinese screening; (3) watch evaluation versions and question-set update dates. This lecture uses no unverified specific scores.

Pros & Limits

Pros: open and reproducible, good Chinese coverage, complete dimensions. Limits: some subsets are public questions, open to optimization; scores across versions are not directly comparable.

SuperCLUE: periodic Chinese benchmark

Background

A third-party evaluation system for Chinese LLMs, publishing periodic reports on Chinese general ability spanning semantic understanding, logic, generation, safety and more.

Mechanism

Primarily Chinese-native tasks, scored on multiple dimensions with total and sub-scores; some sub-boards separate closed and open models, showing the relative position of domestic models.

How to Read

Official entrance: `https://www.superclueai.com/`. Points: (1) note the evaluation month and question-set version; (2) distinguish the "total board" from the "open-source sub-board" — they are not interchangeable; (3) some scores are vendor-submitted — check whether "measured" is marked. This lecture uses no unverified specific scores.

Pros & Limits

Pros: native Chinese corpus, frequent updates, full coverage of domestic models. Limits: less methodological disclosure than academic benchmarks; total-score weights set by the institution; historical calibration changes demand careful comparison.

FlagEval and the C-Eval / CMMLU benchmarks

Background

FlagEval, from BAAI, is a multi-dimensional multimodal evaluation platform; C-Eval and CMMLU are widely used academic Chinese subject-knowledge benchmarks.

Mechanism

FlagEval uses a "capability–task–metric" three-layer structure and supports model submission with automatic evaluation; C-Eval and CMMLU cover dozens of subjects in multiple-choice form, testing Chinese knowledge breadth, with public questions.

How to Read

Official entrance: `https://flageval.baai.ac.cn/`; C-Eval and CMMLU via their official papers and repositories. Points: (1) multiple-choice benchmarks contaminate easily — high scores are not strong evidence; (2) fit for a "Chinese basic knowledge" check-up, not for judging agent ability. This lecture uses no unverified specific scores.

Pros & Limits

Pros: clear structure, broad Chinese subject coverage, reproducible. Limits: single multiple-choice form misses generation quality and multi-step reasoning; public questions risk training leakage.

Hugging Face open LLM board and MTEB

Background

Hugging Face is the world's largest model community; the "Open LLM Leaderboard" was long the most-cited board for open models; embeddings are led by the MTEB/MMTEB series.

Mechanism

The open-model board auto-evaluates community submissions on several public benchmarks, grouped by parameter scale; MTEB covers retrieval, classification, clustering, reranking, semantic similarity and more, multilingual, and is the usual basis for RAG selection.

How to Read

Official entrance: `https://huggingface.co/`. Points: (1) the open-model board has gone through version changes and archiving — always cite version and date; (2) for MTEB read the Chinese/multilingual sub-board, not the total; (3) community boards invite small-model gaming — read the groupings. This lecture uses no unverified specific scores.

Pros & Limits

Pros: free, reproducible, broadest coverage of the open ecosystem. Limits: public question sets mean high contamination risk; board maintenance status changes; historical citations easily break.

SWE-bench: The Gold Standard for Coding Agents

Background

Proposed by Princeton and other researchers; tasks are built from real open-source repository issues and their fixing commits; the most recognized coding-agent evaluation.

Mechanism

The model must read the repository, locate the defect and generate a patch, judged by whether the repository's own tests pass; due to noise and difficulty issues in the original set, the community curated a human-verified subset (Verified).

How to Read

Official entrance: `https://www.swebench.com/`. Points: (1) confirm original vs. Verified — not comparable; (2) check whether "test files visible" is allowed — it affects score value; (3) record cost and step count too. This lecture uses no unverified specific scores.

Pros & Limits

Pros: objective and gradable, close to real engineering, clear pass criteria. Limits: skews to the Python ecosystem; questions come from public repos with leakage risk; high scores do not guarantee handling private codebases.

LiveCodeBench and Terminal-Bench

Background

LiveCodeBench evaluates coding with time-sliced contest problems; Terminal-Bench, from Stanford and Lawrence Berkeley Lab among others, evaluates agents in real terminal environments.

Mechanism

LiveCodeBench uses only contest problems published after model training cutoffs, naturally reducing contamination; Terminal-Bench provides containerized terminal environments and task lists, judged by task completion, with new versions per model generation.

How to Read

Official entrances: LiveCodeBench `https://livecodebench.github.io/`; Terminal-Bench `https://www.tbench.ai/`. Points: (1) read LiveCodeBench scores together with their time window; (2) Terminal-Bench has multiple versions — not cross-comparable; (3) both need per-task cost and timeout settings recorded. This lecture uses no unverified specific scores.

Pros & Limits

Pros: explicit anti-contamination design, reproducible tasks, close to real operation. Limits: depends on the board maintainers' question hygiene; environment setup is complex; scores are sensitive to prompts and scaffolding.

τ^2-bench and BFCL: tool calling

Background

τ^2-bench studies task completion under "agent–user–tool" three-way interaction; BFCL (Berkeley Function-Calling Leaderboard) focuses on the correctness of function calls, the earliest tool-calling-specific evaluation.

Mechanism

τ^2-bench runs multi-turn dialogues in simulated environments with domain rules, requiring the agent both to respect business rules and satisfy the user — measuring the "rule-violation rate" directly; BFCL supplies many function definitions and user requests, testing whether calls are syntactically valid, correctly parameterized and executable.

How to Read

Official entrances: τ^2-bench `https://github.com/sierra-research/tau2-bench`; BFCL `https://gorilla.cs.berkeley.edu/leaderboard.html`. Points: (1) tool-calling scores are highly sensitive to function-description wording — not comparable across boards; (2) read both call correctness and redundant-call rate; (3) multi-turn tasks also need violation counts. This lecture uses no unverified specific scores.

Pros & Limits

Pros: hits the core agent ability, quantifies violations, reproducible tasks. Limits: simulated environments differ from real systems; function sets defined by the evaluators, limited coverage; results sensitive to scaffolding.

GAIA, OSWorld and WebArena: end-to-end agents

Background

GAIA, from Meta and others, poses multi-step real tasks "easy for humans, hard for AI"; OSWorld and WebArena provide real operating-system and website environments as agent testbeds.

Mechanism

GAIA requires unique, gradable answers, combining browsing, retrieval and reasoning; OSWorld operates real software (including office suites) in a virtual machine; WebArena replicates typical website functions for end-to-end goals. All three score by task success rate.

How to Read

Official entrances: GAIA `https://huggingface.co/spaces/gaia-benchmark/leaderboard`; OSWorld `https://os-world.github.io/`; WebArena `https://webarena.dev/`. Points: (1) environments are stateful — note retry rules and step caps; (2) success rates vary hugely with scaffolding — read implementation details; (3) study the "failure mode" distribution, not just the total. This lecture uses no unverified specific scores.

Pros & Limits

Pros: closest to real usage, exposes weaknesses in long chains. Limits: expensive to run, hard to reproduce; public environments invite targeted adaptation; results highly sensitive to the tool set.

Multimodal and speech: MMMU-Pro, Video-MME, DocVQA

Background

Multimodal evaluation comes from several academic and community efforts: the MMMU series tests college-level multidisciplinary image–text understanding, Video-MME video understanding, DocVQA and OCRBench document and text recognition; on speech, the HF Open ASR board and TTS Arena lead.

Mechanism

MMMU-Pro raises question difficulty to reduce "guessing from text cues"; video benchmarks stratify by duration and domain; document benchmarks grade field-level accuracy; speech boards use word error rate (WER) as the main metric and report real-time factor (RTFx) for speed.

How to Read

Official entrances: MMMU `https://mmmu-benchmark.github.io/`; Open ASR `https://huggingface.co/spaces/hf-audio/open_asr_leaderboard`. Points: (1) for documents read field-level accuracy, not overall similarity; (2) for speech weigh WER against real-time factor; (3) video scores are sensitive to sampled frame counts. This lecture uses no unverified specific scores.

Pros & Limits

Pros: clear metrics, close to concrete functional modules. Limits: high single scores do not add up to end-to-end usability; real documents carry far more layout noise than question sets; speech varies sharply with accent and noise.

Generation boards and enterprise services

Background

On generation, Artificial Analysis's image and video arenas and LMArena's text-to-image arena are the main public boards; on the enterprise side, a cluster of vendors focus on hallucination detection and observability.

Mechanism

Generation boards produce Elo rankings from human preference votes, supplemented by automatic metrics (prompt following, aesthetics, text rendering); enterprise services offer hallucination detection, retrieval-quality scoring (faithfulness and relevance for RAG), prompt regression testing and live monitoring.

How to Read

Official entrances: text-to-image arena `https://artificialanalysis.ai/text-to-image/arena`; video arena `https://artificialanalysis.ai/video/arena`. Points: (1) generation quality is highly subjective — check hard metrics like prompt following and text rendering; (2) enterprise services fit ongoing monitoring, not one-off selection; (3) your own golden samples remain the closest fit to your needs. This lecture uses no unverified specific scores.

Pros & Limits

Pros: covers both generation quality and live risk, filling static boards' gap. Limits: strong subjectivity, question sets differ across arenas; commercial scoring criteria are not public, hard to compare.

Portal quick reference (I): indexes and arenas

Institution / board	Official entrance (full URL)	What to check first
LMArena	https://lmarena.ai/	style control on or off; confidence-interval overlap
Artificial Analysis	https://artificialanalysis.ai/	index version; reasoning mode; capability-per-dollar view
Stanford HELM	https://crfm.stanford.edu/helm/	per-dimension metrics, not a single total
Epoch AI	https://epoch.ai/	trend curves; private question-set notes
LiveBench	https://livebench.ai/	update month; per-category scores
ARC Prize	https://arcprize.org/	dual scoring: accuracy and cost per problem
Scale AI (SEAL)	https://scale.com/leaderboard	sub-board confidence intervals; expert authorship notes
Vals AI	https://www.vals.ai/	whether the domain matches your industry
OpenCompass	https://opencompass.org.cn/	composite vs. sub-boards side by side
SuperCLUE	https://www.superclueai.com/	evaluation month; total vs. open-source sub-board

Source: URLs defer to official sites; re-check and screenshot-archive them once per semester before class.

Portal quick reference (II): coding and agents

Institution / board	Official entrance (full URL)
FlagEval (BAAI)	https://flageval.baai.ac.cn/
SWE-bench (repo-level repair)	https://www.swebench.com/
LiveCodeBench (clean coding)	https://livecodebench.github.io/
Terminal-Bench (terminal tasks)	https://www.tbench.ai/
τ^2-bench (multi-turn tool calls)	https://github.com/sierra-research/tau2-bench
BFCL (function calling)	https://gorilla.cs.berkeley.edu/leaderboard.html
GAIA (multi-step real tasks)	https://huggingface.co/spaces/gaia-benchmark/leaderboard
OSWorld (OS operation)	https://os-world.github.io/
WebArena (web operation)	https://webarena.dev/
MMMU (multimodal understanding)	https://mmmu-benchmark.github.io/
MTEB (embedding and retrieval)	https://huggingface.co/spaces/mteb/leaderboard
Open ASR (speech recognition)	https://huggingface.co/spaces/hf-audio/open_asr_leaderboard
AA text-to-image arena	https://artificialanalysis.ai/text-to-image/arena
AA video arena	https://artificialanalysis.ai/video/arena

Source: All URLs are official entrances; some sites may require a proxy on campus networks (e.g. Hugging Face).

Judging an institution's credibility

Before trusting any board, ask six questions.

- Independence: who funds it? does it charge the evaluated parties?
- Transparency: are questions, prompts and parameters public?
- Reproducibility: can outsiders re-run and get similar results?
- Cadence: how often does it update? do question sets roll over?
- Coverage and bias: which languages, domains and task forms are covered?
- Statistical rigor: are confidence intervals and sample sizes given?

Key Terms

Independence
Transparency
Reproducibility
Confidence Interval

Diagram | The six credibility questions: independence · transparency · reproducibility · cadence · coverage and bias · statistical rigor. Missing three or more, treat the board as reference only.

Why the Same Model Ranks Differently Across Boards

Rank gaps usually are not lies — they are methodology gaps.

- Different question sets: some lean academic, some lean real tasks.
- Different configurations: reasoning on or off, thinking budget size, tools allowed or not.
- Different grading: human preference votes, automatic scoring, or test-suite passes.
- Different times: model releases lag board snapshots, and new models often sit "untested".

Key Terms

Methodology Gap
Snapshot Lag
Config Sensitivity
Evaluation Ecosystem

Example | The same model can be No. 1 on board A (human preference), fifth on board B (objective questions) and absent from board C (reasoning mode off at evaluation time). All three are true — different methodologies.

Class worksheet: your own comparison table

Model family	Capability	Price tier	Speed	Context	Source and date
GPT series					
Claude series					
Gemini series					
Qwen series					
DeepSeek series					
Llama series					
Mistral series					

How to use: before class, copy that week's data from official boards into the table; every cell must carry "source + date" or the number may not be used in decisions.

07

How to Read a Leaderboard Without Being Misled

Three Levels of Evaluation

Capability benchmarks, human preference and business acceptance answer three different questions.

- Capability benchmark: "can it get standard questions right" — reproducible and comparable.
- Human preference: "which answer do users like" — reflects experience, not correctness.
- Business acceptance: "is it usable on our golden samples" — closest to need, but not comparable across organizations.
- All three matter: benchmarks shortlist, preference informs experience, your own evaluation decides.

Key Terms

Capability Benchmark
Human Preference
Golden Set
Acceptance Criteria

Diagram | Three levels: benchmarks answer "can it be right", preference answers "is it liked", acceptance answers "can we use it". None substitutes for another.

Anatomy of scoring mechanisms

Scoring	How it is computed	Its weakness
Accuracy / pass rate	share answered or completed	question difficulty sets discrimination; public sets contaminate
Elo / Bradley–Terry	relative strength fitted from pairwise wins	relative preference only, not absolute correctness; voter demographics matter
Weighted composite	several benchmarks merged by weights	weights set by the institution; new weights can crown a new winner
Efficiency scoring	cost or compute enters the denominator	cost accounting (retries, caching) often inconsistent
LLM judge	another model scores the answers	position bias, length bias, self-preference
Expert grading	licensed professionals judge	costly, small samples, inter-rater agreement needs testing

Source: The mechanism notes are general methodology; a single evaluation by one institution may combine several scoring modes.

Pitfall 1: Data Contamination and Benchmark Gaming

Once a question appeared in training data, the score no longer measures capability.

- Data contamination: test questions or close variants entered the training corpus.
- Signature: abnormally high scores that collapse when questions are rephrased; extreme sensitivity to wording.
- Mitigations: private sets, time slicing (only post-cutoff questions), rolling updates.
- Diagnostic: does the model show the same level on other independent benchmarks?

Key Terms

Data Contamination
Time-sliced
Private Held-out Set
Benchmark Gaming

Example | Contamination signature: board scores abnormally high, but errors appear when questions are reworded; extreme sensitivity to phrasing and option order. Then high scores no longer mean capability.

Pitfall 2: Prompt and Configuration Sensitivity

The same model can swing by more than ten points on a prompt change.

- Prompt wording, example counts, and whether step-by-step reasoning is requested all move scores.
- Reasoning mode on or off is two different cost–capability tiers.
- Tools allowed or not, step limits, timeouts — all change results.
- Reading move: find the configuration note first; scores without one are not comparable.

Key Terms

Prompt Sensitivity
Configuration Disclosure
Thinking Budget
Comparability

Example | Same model, same question: plain prompt 68, add "reason step by step" 79, reasoning mode on 84 — three true numbers that must never be mixed.

Pitfall 3: Averages Hide Distributions

Two models with equal totals may be exact complements across tasks.

- Composite scores are weighted averages; extremes get hidden — some categories may be near unusable.
- Read sub-score radars or sub-boards for systematic weaknesses (multilingual, long text, tool calling).
- For a concrete task, "high average" matters less than "high and stable on that sub-score".
- Example: strong writing but weak structured extraction — unfit for document processing.

Key Terms

Sub-score
Radar Chart
Systematic Weakness
Task Fit

Example | Total 80: coding 92, multilingual 55. If your task is multilingual, the total misleads — read the sub-scores and the shortest plank.

Pitfall 4: significance and confidence intervals

A one-rank gap may be pure noise.

- Arena boards publish confidence intervals; when they overlap, "stronger" cannot be claimed.
- Sample size, question count and random seeds all add variance.
- Reporting medians and multi-run results is more honest than one best run.
- Class criterion: "would a re-run flip the ranking? A flip-prone conclusion does not go into the report."

Key Terms

Confidence Interval
Statistical Significance
Variance
Repeatability

Statistical | One rank apart with overlapping intervals: the correct statement is "statistically indistinguishable", not "model A beats model B". Writing that sentence wrong sinks the credibility of the whole report.

Pitfall 5: Inconsistent Cost Accounting

"Price per million tokens" and "cost per task" are two different numbers.

- A cheap unit price with verbose output and retries can cost more in total.
- Reasoning tokens are billed too, invisibly inflating cost.
- Cache hits, batch discounts and free quotas all shift real spend.
- Fair comparison: fix the task and acceptance criteria, measure "cost per completed task".

Key Terms

Token Efficiency
Cost per Task
Cache Hit
Batch Discount

Formula | Cost per task $= T_{\text{in}}P_{\text{in}} + T_{\text{out}}P_{\text{out}} + C_{\text{think}} + C_{\text{tool}} + C_{\text{retry}}$. Only after converting unit prices into this number are different tiers truly comparable.

Pitfall 6: Treating a Leaderboard as a Conclusion

Leaderboards are screening tools, not decision bases.

- Boards measure averages; your needs carry specific constraints and failure costs.
- Board updates lag releases; new models often lack comparable data.
- Boards do not measure: data compliance, supply stability, terms of service, latency, rate limits.
- Right process: shortlist 3–5 on boards → evaluate on your golden samples → shadow deployment → periodic regression.

Key Terms

Shortlist
Shadow Deployment
Regression Test
Decision Basis

Example | A copyable citation: "According to *institution*, *date* (configuration: reasoning on, no tools), this family ranks near the top; actual effect subject to our own golden-sample evaluation."

The Minimal Viable In-House Evaluation

Without heavy engineering you can get more reliable conclusions than any public board.

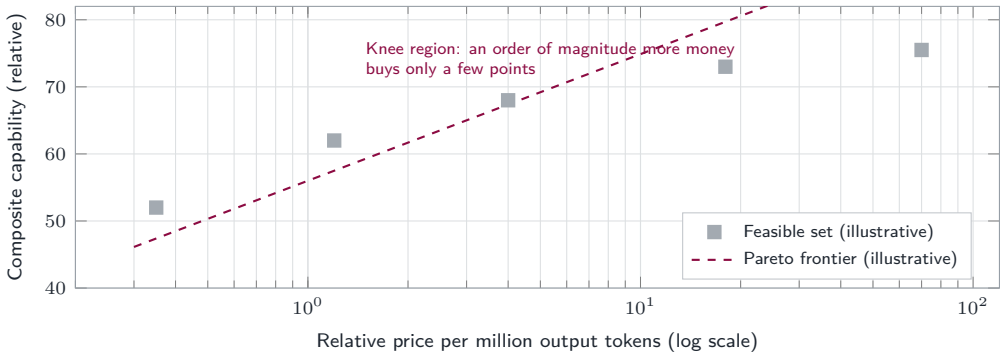
- Golden set: 20–50 real tasks covering typical and extreme cases.
- Blind review: generator and reviewer separated, model identities hidden, confirmation bias reduced.
- Rubric: decompose "good" into gradable dimensions — facts, completeness, format, risk.
- Regression: after every model or prompt change, re-run the same set and watch for regressions.

Key Terms

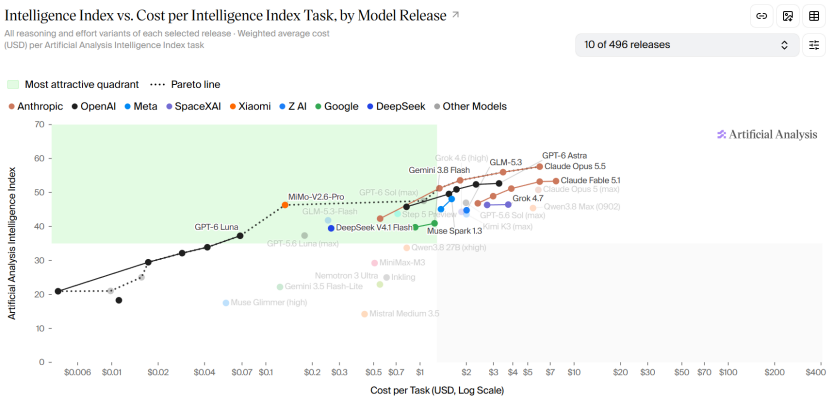
Golden Set
Blind Review
Rubric
Regression Test

Example | 20 golden samples + blind review + one rubric: usually an afternoon of work, yet it answers "is it usable" better than ten public boards.

Pareto frontier: choosing the knee model



Real data: “capability commoditization” by release date



Same coordinate system, colored by model release date: newer versions keep moving left at the same capability level — the per-task cost of equal capability keeps falling.

Source: Artificial Analysis: Intelligence Index vs. Cost per Intelligence Index Task by model release. Screenshot snapshot 2026-09; original annotation by the institution — cite institution and date.

08

From Per-Million-Token Price to Per-Task Cost

Billing Anatomy: What Is Actually on the Bill

Unit price is only the first line of the bill, far from all of it.

- Input tokens: everything you send, including system prompts, retrieved evidence and history.
- Output tokens: what the model writes — usually 3–5x the input price.
- Reasoning tokens: internal computation before the answer, billed as well.
- Add-ons: cache read/write, batching, image and audio tokenization, tool calls, retrieval and reranking.

Key Terms

Input Tokens
Output Tokens
Reasoning Tokens
Tokenization

Diagram | Bill composition: input tokens · output tokens · reasoning tokens · cache · tools and retrieval · failed retries. Most budgets only estimate the first two.

Billing Elements and Their Classic Traps

Element	Typical pricing	Easy-to-hit trap
Input tokens	per million; cheaper than output	long system prompts and full retrieval results re-billed on every call
Output tokens	several times input price	a "talkative" model scales cost directly; format redundancy is cost too
Reasoning tokens	at output price or a special tier	invisible, uncontrollable; harder tasks cost more and are hard to predict
Cache	write expensive, read cheap (a small fraction of input price)	caches have TTLs and expire; prefix changes invalidate them
Batch	discount for async jobs, often about 50% of sync	high latency, offline only; some vendors keep separate quotas
Multimodal input	images tokenized by size and resolution; audio by duration	high-res images can tokenize to thousands of tokens, underestimating cost
Tools and retrieval	per call, per volume, or as extra tokens	multi-turn agent loops multiply cost

Source: Accounting bases and discounts vary by vendor — defer to official pricing pages; this table explains structure, not quotes.

The Cost Formula: Convert Unit Price into Task Cost

The cost of one task equals all tokens plus all calls.

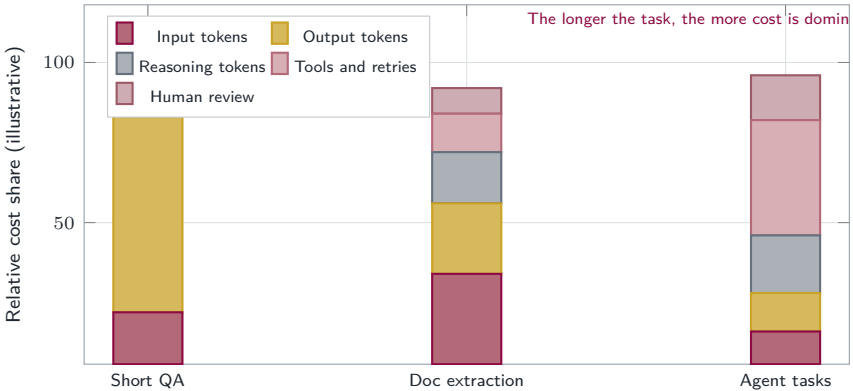
- Token cost: $C_{\text{tok}} = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}}$.
- Full cost: $C = C_{\text{tok}} + C_{\text{cache}} + C_{\text{tool}} + C_{\text{retry}} + C_{\text{human}}$.
- Token efficiency: finishing the same task with fewer tokens is a direct price cut.
- Key corollary: optimizing prompts and context structure often beats switching to a cheaper model.

Key Terms

Token Cost
Retry Cost
Human Review Cost
Token Efficiency

Formula | $C = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}}$. Example: 3,000 input tokens and 800 output tokens, each times its own price — the output price is usually 3–5x the input's.

Cost structure of one task (illustrative)



Caching and batching: neglected discounts

With the same model, the right strategy can cut the bill several-fold.

- Prompt caching: cache the invariant prefix (system prompt, long documents, product manuals); repeat calls pay the low read price.
- The key is a "stable prefix": inserting changing content early in the prompt invalidates the whole cache.
- Batch API: submit non-realtime jobs asynchronously for a significant discount.
- Selection rule: real-time interaction uses sync + caching; offline volume uses batching.

Key Terms

Prompt Caching
Stable Prefix
Asynchronous Batch
TTL

Example | Put the system prompt and long documents at the very front, unchanged, and the cache hits; insert one ever-changing sentence in the middle and the whole cache misses.

The Cost Elasticity of Reasoning Models

The thinking budget is the new cost dial — and the hardest to predict.

- Reasoning models think longer before answering; thinking tokens bill at output price.
- Low/medium/high thinking tiers can differ several-fold in cost on the same question.
- Cost elasticity: running easy questions at the highest tier is common waste.
- Practice: route by difficulty first — light tiers for simple tasks, high budgets only for complex ones.

Key Terms

Cost Elasticity
Thinking Budget
Model Routing
Difficulty Routing

Example | The same hard problem: low tier finishes in 2 seconds at low cost; the highest tier takes 20 seconds at several times the cost, for a few points of accuracy. Who gets the budget depends on the cost of being wrong.

Three Shapes of Total Cost of Ownership

Pay-per-use, self-hosting and hybrid have completely different cost curves.

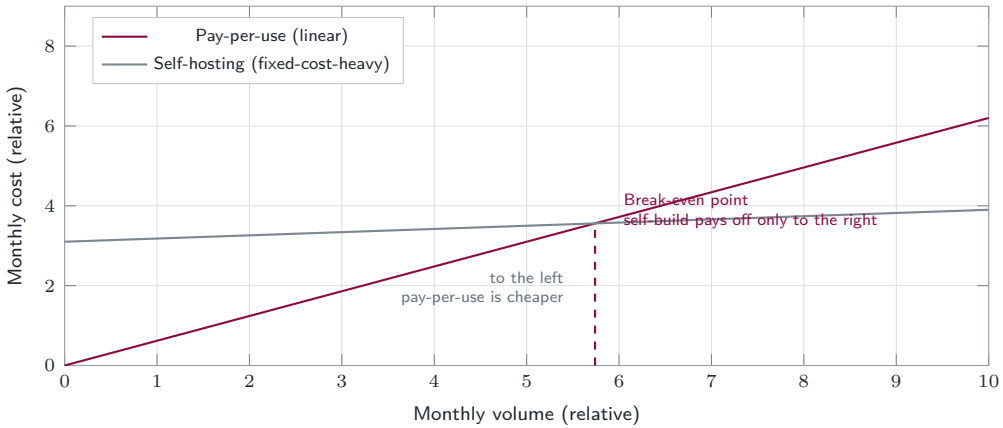
- Pay-per-use: marginal cost grows linearly, near-zero start — fits uncertain demand.
- Self-hosting: mostly fixed cost (GPUs, facility, power, ops) — fits stable, large-scale load.
- Hybrid: routine traffic on cheap self-hosting or light tiers; peaks and complex tasks go to the API.
- The criterion is not "which is cheaper" but "is demand stable, is data sensitive, does the team have ops capability".

Key Terms

Total Cost of Ownership
Marginal Cost
Fixed Cost
Hybrid Deployment

Diagram | Cost curves: self-hosting is fixed-cost-heavy (gentle slope, high start); API is marginal-cost-heavy (near-zero start, steep slope). The crossing point is the break-even point.

Break-even: self-hosting vs. pay-per-use



Self-hosting cost checklist

Cost item	Notes	How often underestimated
Accelerators	purchase or hourly cloud rental; inference needs VRAM for weights and cache	high: VRAM, not compute, is usually the bottleneck
Servers and facility	machines, network, racks, cooling, redundant power	medium: often written off as "existing resources"
Power and cooling	electricity and refrigeration at high load are not negligible	high: a sizable share over long runs
Ops labor	deployment, monitoring, upgrades, incident response, security patches	very high: invisible but continuous
Utilization loss	load swings idle the hardware, raising real unit cost	high: sizing for peak is the least economical
Engineering work	inference frameworks, quantization, batch scheduling, model-update pipeline	medium-high: one-off but hits delivery time

Rule of thumb: self-host only when demand is stable, data is sensitive, and the team has ops capability — all three at once.

Hidden Costs: The Parts off the Bill

The real cost gap often is not in the unit price.

- Latency cost: 3 extra seconds can directly dent willingness to use and conversion.
- Error cost: each extra point of error rate can multiply human review and rework.
- Switching cost: once prompts, evaluations, toolchains and processes bind, migration is expensive.
- Compliance cost: data-residency assessment, audits and contract negotiation all take effort.

Key Terms

Latency Cost
Error Cost
Switching Cost
Compliance Cost

Example | Three extra seconds, one point more error rate, one migration project — together these usually cost far more than the unit-price gap between models.

Six Ways to Cut Cost, Ordered by Payoff

Do the free optimizations first; consider changing models later.

- 1. Context pruning: drop redundant history and irrelevant retrieval — lower input tokens directly.
- 2. Prompt caching: stabilize the fixed prefix so repeats hit cache pricing.
- 3. Model routing: light tiers for simple tasks; flagships only where needed.
- 4. Semantic caching: return stored answers for identical or near-identical requests — no model call at all.
- 5. Structured output: constrain format and length; fewer wasted output tokens.
- 6. Batching and quantization: batch offline jobs; quantize self-hosted models to cut VRAM.

Key Terms

Semantic Cache
Model Routing
Quantization
Output Constraint



Three Ways to Misread a Price Sheet

The three easiest mistakes to make with a quote.

- Misreading 1: comparing input prices only — output prices are several times higher, and output volume is task-determined.
- Misreading 2: ignoring reasoning tokens — a reasoning model's effective price can far exceed its nominal output price.
- Misreading 3: comparing across currencies and units — USD vs. CNY, per-million vs. per-thousand must be unified first.
- Correct move: convert the sheet into "cost of completing this task of mine", then compare.

Key Terms

Unit Normalization
List vs. Effective Price
Exchange Rate
Cost per Task

Example | Input 0.1 and output 0.4 look cheap, but when output volume is 5x input, cost is dominated by the output price. Estimate your own input/output ratio before comparing.

Class Worksheet: Convert a Price Sheet into Task Costs

Model tier	Input price	Output price	Est. tokens	Cost per run	Notes (cache / batch / reasoning tokens)
Flagship					
Mainstream					
Light					
Reasoning flagship					
Open weights, self-hosted					

Requirements: prices must state currency and unit (e.g. USD / per million tokens) and the source date; cost per run computed as $C = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}}$.

09

Thirteen Directions and Their Sweet Spots

Comparison framework: metrics per direction

Direction	Primary capability metric	Primary cost basis	Main board type
Flagship general	composite index, arena Elo	per-million tokens, per task	composite + arena
Reasoning models	math and contest-grade reasoning	cost elasticity from thinking tokens	academic + composite
Light models	capability per dollar	unit price and throughput	composite value view
Open-weights models	relative capability at equal scale	self-hosting TCO	community boards + independent measurement
Code and coding agents	task pass rate, repo-level repair	per-task cost and steps	agent-specific
Multimodal and documents	field-level accuracy, chart understanding	tokenized image cost	multimodal-specific
Speech	WER, subjective preference	per-hour audio cost	speech-specific
Image generation	prompt following, text rendering	per-image cost	generation arenas
Video generation	temporal consistency, duration	per-second video cost	generation arenas
Embedding and reranking	retrieval recall	per-million tokens (very low)	embedding-specific
Chinese and multilingual	Chinese understanding and generation	same as general	Chinese regional
On-device and privacy	post-quantization capability retention	one-off hardware + power	community measurement + self-test
Aggregators and providers	availability, routing, rate limits	markup and minimum spend	platform disclosures + self-test

How to use: locate the direction first, then its metrics and board type, and finally confirm with your own golden samples.

Direction 1: Flagship General Models

Metrics

Composite intelligence index, arena preference Elo, per-domain scores (coding, math, writing, hard prompts); context window and effective usable length; multilingual coverage.

Cost

Input and output prices, whether reasoning tokens are billed separately, cache and batch discounts; convert "per million tokens" into "cost of one run of my task".

Where

LMArena (human preference) and Artificial Analysis (measured index, speed, price) are the two main lines; their conclusions can differ — read both.

Selection

Flagships excel at complex reasoning and long chains; for short classification and simple extraction, the flagship–light gap is often smaller than the price gap. Flagships are the reference ceiling — benchmark, not default.

Direction 2: Reasoning Models

Metrics

Math and contest problems, scientific reasoning, complex code, planning; behind high scores, always check the thinking budget used.

Cost

Thinking tokens bill at output price or a special tier; the same question across thinking tiers can differ several-fold — quote per tier.

Where

Academic benchmarks (expert math, contest problems) and composite boards; note whether reasoning mode was on, and whether tools were allowed.

Selection

Fit multi-step, verifiable, wait-tolerant tasks; unfit for high-frequency simple QA. Diminishing returns are clear: doubling the thinking budget usually buys far less accuracy than it costs. Always cap the budget.

Direction 3: Light Models and the Low-Cost Tier

Metrics

Capability per dollar, throughput, stability; accuracy on classification and extraction tasks.

Cost

Low unit price is the surface: check whether accuracy under few-shot prompting suffices, and whether retries multiply.

Where

The value view of composite boards and "capability per dollar" rankings on independent measurement platforms; community boards for same-scale comparisons.

Selection

High-value uses: batch preprocessing (classification, tagging, first-pass screening, draft translation), the first hop of model routing, and low-sensitivity edge tasks. Weaknesses: complex instruction following and long context, failing on boundary samples.

Direction 4: Open-Weights Models

Metrics

Relative capability at equal scale, post-quantization capability retention, inference throughput and VRAM; maturity of the fine-tuning ecosystem.

Cost

Self-hosting TCO: accelerators, facility, power, ops, utilization loss; compare against pay-per-use for the break-even point.

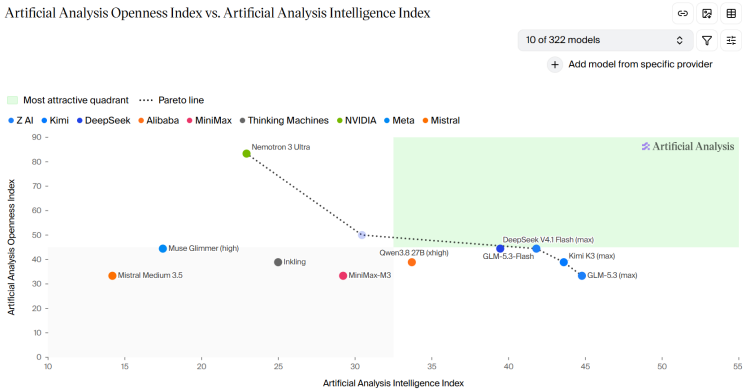
Where

Community open-model boards (mind versions and dates), open-weights groupings on independent platforms, and per-parameter-scale groupings.

Selection

Core value is control and bargaining power: private deployment, fine-tuning, running without external services. The price is engineering and ops responsibility. Verify license terms per version (commercial use, naming, thresholds).

Real data: Openness Index vs. Intelligence Index



Vertical axis: openness index; horizontal axis: intelligence index. Open-weights models (DeepSeek, GLM, Kimi, etc.) have entered the high-intelligence region, but openness and capability do not advance in lockstep.

Source: Artificial Analysis: Openness Index vs. Intelligence Index. Screenshot snapshot 2026-09; original annotation by the institution — cite institution and date.

Direction 5: Code and Coding Agents

Metrics

Repo-level repair pass rate, terminal task completion, contest pass rate; also record steps and cost.

Cost

Billing per completed task is fairer: includes retries, test runs and failed attempts; longer context means higher cost.

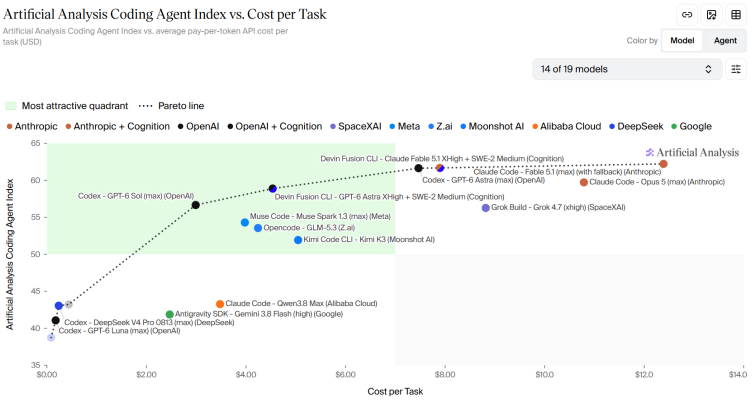
Where

Agent-specific boards: repo-level repair, terminal tasks, time-sliced contest benchmarks; mind dataset versions on all of them.

Selection

The most thoroughly validated direction, thanks to objective test feedback. Boundary: private codebases, non-mainstream languages and legacy systems need your own evaluation.

Real data: Coding Agent Index vs. cost per task



Coding agents must be read on two axes at once: task quality and cost per task. Reading only the index ignores costs that can differ by an order of magnitude.

Source: Artificial Analysis: Coding Agent Index vs. Cost per Task. Screenshot snapshot 2026-09; original annotation by the institution — cite institution and date.

Direction 6: Multimodal and Document Understanding

Metrics

Field-level accuracy (not overall similarity), robustness to complex layouts, chart and table understanding, screen-operation ability.

Cost

Images tokenize by resolution — a large high-res image can cost thousands of tokens; video and audio by duration.

Where

Multimodal-specific benchmarks (college-level image–text, long-video understanding, document QA, OCR) and the vision sub-boards of composite indexes.

Selection

A model strong at general vision is not automatically reliable on receipts, handwriting and low-quality scans. Critical fields need rule checks or human review; "recognized most of the text" is not "extracted correctly".

Direction 7: speech and real-time dialogue

Metrics

Recognition: WER and real-time factor (RTFx); synthesis and dialogue: subjective preference, naturalness, response latency.

Cost

Billed by audio duration (per minute or hour); real-time dialogue, with two-way streaming, differs in cost structure from offline transcription.

Where

HF Open ASR board (macro-average over datasets + real-time factor); speech-synthesis arenas; the speech sub-scores of composite boards.

Selection

Always self-test in your own scenario (accents, noise, jargon, multi-speaker): clean-audio board scores usually beat real calls markedly. Real-time dialogue also needs interruption handling and end-to-end latency evaluated.

Direction 8: Image Generation

Metrics

Human preference Elo, prompt following, text-rendering correctness, composition and consistency; stability across batch generation.

Cost

Per image, varying with resolution and quality tier; re-rolls and iterations multiply cost.

Where

Generation arenas (human preference) plus automatic metrics (prompt following, aesthetics, text rendering).

Selection

Preference rankings are the most subjective, and arenas differ in question sets and voter pools — do not compare across them. For commercial use, additionally confirm training-data and copyright terms, and commercial licensing.

Real data: Text-to-image leaderboard

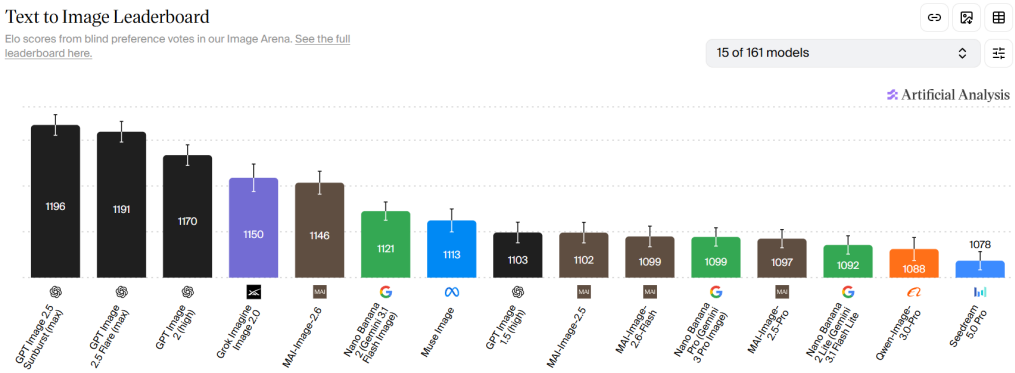


Image generation ranks by human preference votes — the most subjective of all; before commercial use, confirm training-data and copyright terms.

Source: Artificial Analysis: Text to Image Leaderboard. Screenshot snapshot 2026-09; original annotation by the institution — cite institution and date.

Direction 9: Video Generation

Metrics

Duration and resolution, temporal consistency (jitter or deformation), physical plausibility, camera control.

Cost

Per second or per run, markedly above images; regeneration is expensive — draft at low resolution first.

Where

Video-generation arenas (human preference) and automatic benchmarks (temporal consistency, semantic alignment).

Selection

Current bottlenecks are temporal consistency and controllability, not single-frame quality. Fits short clips and concept demos; long-form narrative still needs human editing and stitching.

Direction 10: Embedding and Reranking Models

Metrics

Retrieval recall (Recall@k), ranking quality (nDCG), multilingual and long-text support, vector dimension and storage cost.

Cost

Unit price is tiny but not zero: vectorizing a whole knowledge base is one-off, and every query embeds too.

Where

MTEB/MMTEB retrieval and reranking sub-boards — read the Chinese or multilingual sub-board, not the total.

Selection

The embedding model caps retrieval quality — the most overlooked, most consequential link in a RAG system. Embedding and reranking must be chosen as a pair: reranking lifts precision but adds latency.

Direction 11: Chinese and Multilingual Ability

Metrics

Chinese understanding and generation quality, terminology and register appropriateness, coverage and stability of smaller languages; mixed Chinese–English handling.

Cost

Chinese tokenization efficiency affects cost: the same passage can tokenize very differently across schemes.

Where

Chinese regional boards (composite + sub-boards), Chinese academic benchmarks, and multilingual sub-boards.

Selection

International models' Chinese has improved fast, but idioms, officialese, mixed text and long Chinese documents remain differentiators. Evaluate per target language — strong Chinese–English does not extrapolate to smaller languages.

Direction 12: on-device and privacy

Metrics

Post-quantization capability retention, first-token latency and token speed, memory and power, offline availability.

Cost

One-off hardware plus power; marginal cost near zero — at the price of a lower capability ceiling and slower updates.

Where

Community small-model boards grouped by parameters, official benchmarks of on-device inference frameworks, and your own task tests.

Selection

Fits fixed narrow tasks (classification, extraction, rewriting, offline transcription) and data that must never leave. Never assign complex reasoning or long-context tasks to an on-device model; verify capability drop per quantization level.

Direction 13: Aggregators and Inference Providers

Metrics

Availability and rate-limit policies, breadth of model coverage, failover, data-retention policy, billing and invoicing compliance.

Cost

Markups, minimum spends or quota bundles exist; also weigh "integration cost saved" against "one more dependency".

Where

The platform's own status and pricing pages (labeled as proxy information), platform comparisons on independent measurement sites, and your own load tests.

Selection

Aggregators lower comparison and switching costs; the risk is a new middle-layer dependency and extra data-retention clauses. In production, keep a direct-connection fallback.

Worksheet 1: general and reasoning tiers

Model family	Index	Arena Elo	Speed	In/Out price	Source and date
Flagship general (family 1)					
Flagship general (family 2)					
Reasoning flagship					
Mainstream tier					
Light tier					

Worksheet 2: open weights, coding, multimodal

Model family	Repo repair	Doc extraction	Post-quant.	Self-host cost	Source and date
Open weights (family 1)					
Open weights (family 2)					
Coding-specialized					
Multimodal flagship					
Speech models					

Worksheet 3: generation, embedding, Chinese

Model family	Gen. preference	Prompt follow.	Retrieval recall	Chinese quality	Source and date
Image generation (family 1)					
Video generation (family 1)					
Embedding model					
Reranking model					
Chinese-optimized model					

Three Disciplines of Side-by-Side Comparison

Hold these three, and the conclusion stands.

- One comparison must fix: task, prompt, acceptance criteria and time window.
- Every number must trace to "which institution, which day, what configuration".
- Conclusions must declare boundaries: applies to what, not to what, sample size how large.

Key Terms

Controlled Variables
Traceability
Scope of Validity
Sample Size

Example | The three disciplines inverted: change the model without changing the prompt, numbers without source dates, conclusions without boundaries — such a comparison can be re-verified by no one, and has no decision value.

10

From Leaderboard to Decision: An Auditable Process

The Seven-Step Selection Method

Turn the unanswerable "which model is best" into a series of answerable questions.

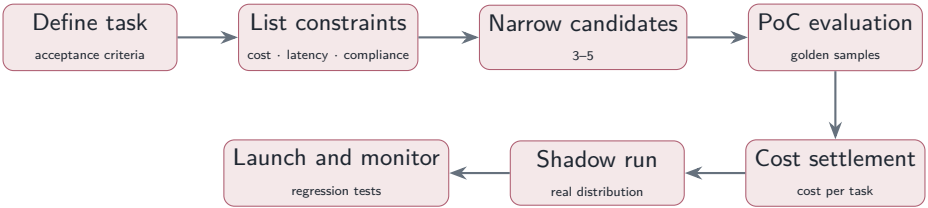
- 1. Define the task and acceptance evidence: what counts as done, who judges?
- 2. List constraints: cost ceiling, latency, compliance, deployment.
- 3. Narrow candidates (3–5) with leaderboards and classification.
- 4. Design the PoC: golden samples, rubric, blind review.
- 5. Settle costs per completed task, not unit price alone.
- 6. Shadow-run on small traffic; watch the real distribution.
- 7. After launch: regression tests, monitoring, periodic re-evaluation.

Key Terms

Acceptance Evidence
Proof of Concept
Shadow Deployment
Continuous Evaluation



The Selection Process: Every Step Has a Concrete Deliverable



Four classes of constraints: write them down first

Constraints are negotiable — but first fix which are not.

- Hard constraints: data must not leave our control; must be privately deployed; must support a given language.
- Soft constraints: desired latency bound, expected cost range.
- Implicit constraints: can the team operate it, is there already a cloud agreement, procurement lead time.
- Payoff of writing them down: the candidate set usually collapses from dozens to three to five.

Key Terms

Hard Constraint
Soft Constraint
Implicit Constraint
Procurement Cycle

Example | "Data must not leave our control" is a hard constraint that eliminates every overseas closed cloud API at once. Write constraints down first — it saves a mountain of pointless model comparisons.

Weighted scorecard: auditable scoring

Dimension	Weight	Cand. A	Cand. B	Cand. C	Evidence (must be verifiable)
Task accuracy	30%				golden-sample pass rate
Factual reliability	20%				hallucination rate, citable share
Cost per task	15%				settled by formula, retries included
Latency and throughput	10%				first-token latency, tokens per second
Format compliance	10%				valid structured-output rate
Compliance and deployment	10%				data residency, private-deployment ability
Supply and terms	5%				openness, SLA, exit plan
Total	100%				**Veto items: privacy leak, unauthorized commitment, unauditable**

Weights must be set by the business side, never by the model or the vendor; comparisons are valid only under one weight system.

How to Run a Proof of Concept

The PoC's goal is not "prove it works" but "find out how it fails".

- Sample design: typical, boundary and known-hard samples, each with a share.
- Evaluation: blind, multi-party independent scoring — never "grading your own plan".
- Records: not just right or wrong — failure types, time, cost, human rework.
- Pass criteria fixed in advance: e.g. "accuracy no lower than X, cost per run no more than Y".

Key Terms

Edge Case
Failure Mode
Pass Criteria
Rework

Example | The PoC's value is discovering failure modes: if 3 of 20 samples fail on "invalid output format", the priority is structured output — not switching models.

Vendor and Compliance Risk

The cost of choosing the wrong model is usually smaller than the cost of a broken vendor relationship.

- Terms of service: is your data used for training, how long retained, usable with sensitive data?
- Availability: rate limits, capacity guarantees, incident history, status-page transparency.
- Exit strategy: can weights be downloaded, data exported, how much work to switch?
- Compliance: cross-border data transfer, industry regulation, audit and traceability.

Key Terms
SLA
Data Retention
Exit Strategy
Auditability

Example | One sentence in the contract — "customer data may be used to improve services" — can decide whether the whole plan passes compliance. Confirm such clauses early in selection, not before launch.

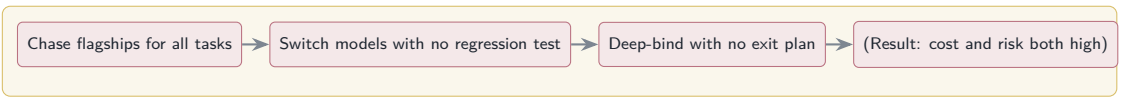
Ten Common Anti-Patterns

These look convenient and bill you six months later.

- 1. Using a flagship for everything; 2. comparing unit prices, not task costs.
- 3. Deciding on one board alone; 4. ignoring latency's damage to experience.
- 5. Tuning the evaluation set and prompt together into overfitting; 6. switching models with no regression test.
- 7. Keeping critical rules only in the prompt; 8. no cost or turn caps.
- 9. Ignoring licenses and data terms; 10. deep binding with no exit plan.

Key Terms

Anti-pattern
Overfitting the Test Set
Turn Limit
Quality Gate



Decision matrix: tier by constraint

Scenario constraint	Recommended tier	Rationale and cautions
High-frequency, simple, batchable	light tier + batching	best unit cost; confirm accuracy on golden samples first
Low-frequency, complex, wait-tolerant	reasoning flagship	trade thinking budget for accuracy; always cap the budget
Data must not leave our control	open weights + on-premise	confirm license and VRAM first, then weigh ops investment
Latency-sensitive, interactive	mainstream + caching	first-token latency matters more than totals here
Multimodal input, field-level precision	multimodal flagship + rule checks	critical fields need a second check — never the model alone
Knowledge-base QA	mainstream + retrieval + reranking	the ceiling is set by retrieval quality, not model scale
Exploration and quick validation	aggregator + free quota	lowers experimentation cost; keep a direct fallback in production

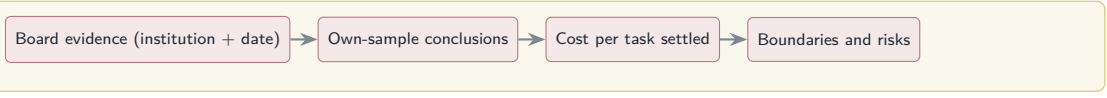
Summary: an auditable evidence chain

In a good selection report, anyone can re-verify the conclusion along the evidence.

- Every conclusion answers: which board, which day, what configuration, how many samples?
- Every cost figure states currency, unit and computation.
- Every recommendation states its boundary of validity and known failure scenarios.
- The final deliverable is not "we recommend model X" but "a continuously updatable evaluation process".

Key Terms

Evidence Chain
Scope
Auditable
Sustainable Evaluation



11

Compare Hands-On, and Know the Limits

Exercise 1: Blind Multi-Model Test on One Task

Task design

- Choose an objectively gradable task (structured extraction or classification) and prepare 10–20 samples.
- Send the same prompt to at least 3 models of different tiers (identities hidden).
- Students exchange reviews and score with the rubric.
- Record: accuracy, failure types, time, cost, human rework.

Item	A	B	C	Notes
Accuracy				
Failure types				
Cost per run				
Avg. time				
Rework				

Discussion: is the most accurate model also the best value? Under what conditions does the conclusion flip?

Exercise 2: Cost Estimation and Break-Even

Task design

- Estimate a business flow's daily/monthly tokens (input, output, reasoning separately).
- Compute monthly cost for three tiers via $C = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}}$.
- Estimate self-hosting fixed costs (GPUs, power, ops) and find the break-even point.
- Discuss: which assumptions are most fragile? Does halving the volume change the conclusion?

Item	Light	Main	Flagship	Notes
Input tokens/mo				
Output tokens/mo				
Monthly cost				
Self-host cost				
Break-even				

Note: prices must state currency, unit and source date; numbers without sources may not enter conclusions.

Exercise 3: board debugging — find misleading spots

Task design

- Open any third-party board and check it against the six pitfalls of Section 7.
- Find at least three potentially misleading spots and write the reasons.
- Write one passage of "correct citation" (institution, date, configuration, limits).
- Cross-check: students challenge each other's conclusions.

No.	Misleading spot found and reason
1	
2	
3	

Risk List: Four Things That Matter More than Capability

Models change; risks do not dissolve on their own.

- Freshness: boards and prices move constantly — every number must carry a date.
- Contamination: public question sets may sit in training data; high scores are not proof of ability.
- Supply: model sunsets, price changes, rate limits and term changes all hit running systems.
- Compliance: data residency, industry regulation and copyright need professional judgment.

Key Terms

Freshness Risk
Contamination Risk
Supply Risk
Compliance Risk

Example | Boards shift monthly, prices adjust anytime, terms change without notice. What deserves investment is "a process refreshed every semester", not any one day's conclusion.

Summary: The Three Artifacts of This Lecture

One map, one table, one formula.

- One map: the five-layer industry stack and three organizational routes (closed / open weights / cloud in-house).
- One table: the per-direction "metrics – cost – entrance" table, plus your own golden-sample results.
- One formula: $C = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}} + C_{\text{cache}} + C_{\text{tool}} + C_{\text{retry}}$ — turn unit price into task cost.
- One reminder: method outlives conclusion — boards change; the way of reading them does not.

Key Terms

Evidence Chain
Cost per Task
Durability of Method
Continuous Update

Formula | The one formula worth memorizing: $C = T_{\text{in}} P_{\text{in}} + T_{\text{out}} P_{\text{out}} + C_{\text{cache}} + C_{\text{tool}} + C_{\text{retry}}$. It translates "price per million tokens" into "cost of one completed task".

Homework

Complete one real comparison with an auditable process.

- 1. Choose an objectively gradable task; prepare 20 golden samples.
- 2. Pick three models of different tiers; run the blind test and fill the rubric.
- 3. Estimate each one's "cost per completed task"; draw a capability–cost scatter.
- 4. Write a one-page selection memo: constraints, evidence, conclusion, boundaries, risk controls.

Key Terms
Golden Set
Rubric
Selection Memo
Scope

Example | Delivery template: one page of constraints (hard/soft list), one page of evidence (boards and self-tests, with dates), half a page of conclusion, half a page of risks and boundaries.

Further Reading and Data Sources

Category	Representative material	Use
Founding architecture	"Attention Is All You Need" (2017)	the original motivation of attention and Transformers
Scale paradigm	GPT-3 paper (2020), scaling-law research	why models were made big
Alignment methods	InstructGPT paper (2022)	instruction tuning and human feedback
Agent paradigm	ReAct paper (2022), Toolformer (2023)	the reason–act loop and tool learning
Protocol	Model Context Protocol (MCP) official spec	the design of tool-interface standardization
Evaluation methods	institutions' methodology pages and annual reports	how the scores are computed
Price and specs	vendors' official pricing pages and model docs	traceable prices and context parameters

Citation discipline: papers — version and year; boards — institution, date and configuration; prices — currency, unit and retrieval date.

Appendix

Glossary and Usage Notes

Appendix A: Glossary (I)

Term	Full form	Term	Full form
LLM	Large Language Model	Agent	Autonomous tool-using assistant
Foundation model	Pre-trained general-purpose model	Tool calling	Structured call to external tools
Token	Smallest text unit	Function calling	Structured API invocation
Context window	Maximum visible token span	MCP	Model Context Protocol
Prompt	Instruction given to the model	Harness	Runtime and governance shell
Context engineering	Arranging everything the model sees	Sandbox	Isolated execution environment
RAG	Retrieval-Augmented Generation	Human in the loop	Human approval inside the loop
Embedding	Dense vector of a text	Prompt injection	Hijacking via embedded text
Reranking	Re-ordering retrieved results	Agent orchestration	Coordinating multiple agents
Fine-tuning	Further training on task data	Multi-agent	Several cooperating agents
Instruction tuning	Training on instruction–answer pairs	Computer use	Operating screen, mouse, keyboard
RLHF	Reinforcement Learning from Human Feedback	Coding agent	Agent that edits and runs code
Distillation	Training small models from large ones	Task success rate	Share of tasks completed
Quantization	Lower-precision weights	Error compounding	Step errors multiply over steps
MoE	Mixture-of-Experts	Benchmark	Standardized test set
Active parameters	Parameters used per token	Leaderboard	Ranked comparison of models
Dense model	All parameters activate	Data contamination	Test data leaked into training
Inference	Running the trained model	Confidence interval	Range of statistical uncertainty

Appendix B: Glossary (II)

Term	Full form	Term	Full form
Reasoning model	Thinks longer before answering	Pareto frontier	Best capability–price boundary
Test-time compute	Inference-stage computation	TCO	Total Cost of Ownership
Chain-of-Thought	Step-by-step intermediate reasoning	Marginal cost	Cost of one extra unit
Thinking budget	Cap on reasoning tokens	Prompt caching	Reusing the stable prefix
Cost elasticity	Cost response to settings	Batch processing	Discounted asynchronous jobs
Hallucination	Fluent but false output	Token efficiency	Task done with fewer tokens
Human preference	What users vote for	Model routing	Assigning tasks to tiers
Arena	Blind pairwise voting board	Semantic cache	Answer reuse for same intent
Bradley–Terry	Win-loss model behind Elo	Vendor lock-in	Dependence on one provider
Golden set	Your own reference samples	Open weights	Downloadable model weights
Rubric	Decomposed scoring criteria	Self-hosting	Running the model yourself
Regression test	Re-run after every change	On-premise	Deployment inside your own estate
Blind review	Identity-hidden evaluation	SLM	Small Language Model
Controlled variables	Fixed comparison conditions	On-device inference	Running on end-user hardware
Transparency	Open questions and methods	Multimodal	Text, image, audio, video
Reproducibility	Others can re-run it	WER	Word Error Rate (speech)
Statistical significance	Gap beyond random noise	RTFx	Real-Time Factor (speech speed)
Vendor benchmark	Self-reported evaluation	SLA	Service Level Agreement

Appendix C: Usage Notes and Update Conventions

How this lecture handles data

- Charts marked "illustrative" show shape and trend only; unmarked charts are official leaderboard screenshots — cite institution and snapshot date.
- Pages touching "latest results" always provide the official entrance and reading points.
- Pages involving "latest results" give official entrance links, so the deck can be refreshed with that week's data before class.
- Models are always named by series to avoid version churn dating the deck.

Per-semester update checklist

- Copy that week's data from official boards into the three worksheets.
- Re-check vendor pricing pages; update currency, unit and dates.
- Re-verify open-weights license terms for changes.
- Check cited boards for redesign or archiving; update entrances.

Update principle: rather leave a blank marked "to be verified" than fill in a number whose source cannot be traced.

Thank You

Next step: with the same task and the same rubric, compare three models of different tiers yourself.